

**[Vecinos Conectados]
(DAS) Documento Arquitectura de Software
Versión 2.7**

Identificación de Documento

Identificación	INF-VC-01
Proyecto	Vecinos Conectados
Versión	2.6

Documento mantenido por	Dairys Bernal, Rocío Bustos y Amaru Burdiles
Fecha de última revisión	21 de junio del 2026
Fecha de próxima revisión	22 de junio del 2026

Documento aprobado por	Dairys Bernal, Amaru Burdiles, Rocío Bustos
Fecha de última aprobación	21 de junio del 2026

Historia de Revisiones

Fecha	Versión	Descripción	Autor
30 de marzo del 2026	1.0	Se desarrollaron los puntos 1.1 al 1.4	Dairys Bernal
01 de abril del 2026	1.1	Se revisaron los puntos anteriores y se ejecutaron los puntos 1.5 al 2.2	Rocío Bustos
03 de abril del 2026	1.2	Se abordaron los puntos 2.3 al 2.5	Amaru Burdiles
05 de abril del 2026	1.3	Se realizo una revisión completa de todos los puntos a abordar del documento	Dairys Bernal
06 de abril del 2026	1.4	Se modifiko formato del informe.	Rocío Bustos
06 de abril del 2026	1.5	Revisión completa	Amaru Burdiles Dairys Bernal Rocío bustos
20 de abril del 2026	1.6	Se agrego una breve descripción a los requerimientos	Dairys Bernal Amaru Burdiles Rocio Bustos
4 de mayo del 2026	1.7	Se desarrollaron los puntos 4.11 y 4.1.2	Dairys Bernal Amaru Burdiles Rocio Bustos
5 de mayo del 2026	1.8	Se realizo caso de uso general y caso de uso especifico, lista de caso de uso	Dairys Bernal Amaru Burdiles Rocio Bustos
11 de mayo del 2026	1.9	Se desarrollo diagrama de clase y diagrama de paquetes	Dairys Bernal Amaru Burdiles Rocio Bustos
12 de mayo del 2026	2.0	Se desarrollo diagrama de actividad, diagrama de componentes y se corrigieron faltas ortográficas	Dairys Bernal Amaru Burdiles Rocio Bustos
16 de mayo del 2026	2.1	Se desarrollo diagrama de despliegue y se corrigieron saltos de línea del documento	Amaru Burdiles Dairys Bernal Rocio Bustos
17 de mayo del 2026	2.2	Se modificaron las descripciones del diagrama de clase y diagrama de componentes	Dairys Bernal Amaru Burdiles Rocio Bustos
16 de junio del 2026	2.3	Se desarrollaron los puntos 5 y 6	Dairys Bernal

17 de junio del 2026	2.4	Se corrigieron los diagramas de casos de uso general y específico, como también el diagrama de clases	Amaru Burdiles Rocío Bustos
17 de junio del 2026	2.5	Se corrigió el prototipo	Amaru Burdiles Rocío Bustos
17 de junio del 2026	2.6	Se desarrollo la evaluación de calidad Heurística de Nielsen	Amaru Burdiles Rocío Bustos
16 de junio del 2026	2.7	Se desarrollo el punto 7 y el control de versión utilizado	Dairys Bernal
19 de junio del 2026	2.8	Se desarrollaron las conclusiones	Dairys Bernal

Tabla de Contenidos

Identificación de Documento	1
Historia de Revisiones	2
Tabla de Contenidos	4
1. INTRODUCCIÓN	4
1.1. Contexto del Problema:	5
1.2. Propósito:	5
1.3. Ámbito:	5
1.4. Definiciones, acrónimos y abreviaciones	5
1.5. Resumen ejecutivo	5
1.6. Arquitectura del sistema	6
2. VISIÓN DEL SISTEMA	6
2.1. Descripción general del sistema	6
2.2. Objetivos del sistema	6
2.3. Requerimientos funcionales	6
2.4. Requerimientos no funcionales	9
2.5. Supuestos y dependencias	10
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	11
3.1. Estilo arquitectónico adoptado	12
3.2. Justificación del estilo según el contexto del sistema	12
3.3. Patrones de diseño aplicados	12
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	12
4.1. VISTA DE ESCENARIO	12
4.2. VISTA LÓGICA	16
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	18
4.4. VISTA DE PROCESOS	20
4.5. VISTA FÍSICA	22
5. REQUISITOS DE CALIDAD	23
5.1. Propósito	24
5.2. Atributos de calidad	24
5.3. Reglas y criterios de evaluación de calidad	24

6. PRINCIPIOS DE DISEÑO APLICADOS	25
6.1. Propósito	25
6.2. Principios de diseño	25
7. PROTOTIPO	26
7.1. Propósito	26
7.2. Mockups (imágenes con una breve descripción)	26
7.3. Justificar herramientas de prototipado	36
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	37
8.1. Propósito	38
8.2. Lista de verificación	38
8.3. Análisis y métricas de resultados	38
9. CONTROL DE VERSIONES	38
9.1. Propósito	39
9.2. Control de versión utilizado	39
9.3. Justificar herramientas de versionamiento	39
10.CONCLUSIONES	40
11. BIBLIOGRAFÍA	41
12. ANEXOS	42
13. CARTA GANTT	42

1. INTRODUCCIÓN

1.1. Contexto del Problema:

El edificio "El Mirador", una estructura de 20 pisos y 160 departamentos situada en una zona céntrica, enfrenta una crisis en la gestión de sus gastos comunes. Actualmente, la administración depende de procesos manuales ineficientes que han derivado en una alta tasa de morosidad, falta de liquidez y opacidad en el uso de los fondos. La desproporción entre el personal disponible y la cantidad de residentes agrava la capacidad de respuesta ante imprevistos, deteriorando la convivencia y la seguridad de la comunidad.

1.2. Propósito:

Este documento tiene como propósito definir el marco arquitectónico para el desarrollo de una solución tecnológica integral. Se busca establecer las directrices técnicas que aseguren la automatización de cobros, la transparencia informativa y la eficiencia operativa, mitigando los riesgos financieros y legales asociados a la gestión actual.

1.3. Ámbito:

El sistema abarcará la gestión automatizada de residentes (propietarios y arrendatarios), el procesamiento de pagos de gastos comunes y extraordinarios, la emisión de notificaciones en tiempo real y la administración de solicitudes o reclamos. Asimismo, integrará un módulo de control y registro de visitas para fortalecer la seguridad perimetral, permitiendo la autorización anticipada y el monitoreo de ingresos. La solución será accesible tanto para la administración como para los residentes a través de una interfaz intuitiva, asegurando una trazabilidad completa de todas las interacciones dentro de la comunidad.

1.4. Definiciones, acrónimos y abreviaciones

ACRONIMO	DESCRIPCION
<i>DAS</i>	Documento de arquitectura de software: Documento técnico que describe el diseño y la estructura del sistema.
<i>GGCC</i>	Gastos comunes: cobros mensuales por servicios y mantención.
<i>CU</i>	Caso de Uso: Descripción de una secuencia de interacciones entre el sistema y sus actores.
<i>UI/UX</i>	Interfaz de usuario/ Experiencia de usuario: Relacionado con el diseño intuitivo requerido para residentes y administración.
<i>SQL</i>	Structure Query Language: Lenguaje utilizado para la gestión y consulta de las bases de datos relacionales del edificio.
<i>PDV</i>	Punto de Venta: Referencia a los centros o mecanismos de recaudación de deudas.
<i>4+1</i>	Modelo de vistas: metodología utilizada para describir la arquitectura mediante cinco vistas (escenario, lógica, desarrollo, procesos y física).

1.5. Resumen ejecutivo

El proyecto propone la implementación de una plataforma digital centralizada que sustituya los procesos manuales por flujos automatizados. Mediante la integración de módulos de cobranza, comunicación y seguridad, se espera reducir la morosidad, optimizar el flujo de caja y restaurar la confianza de los residentes a través de un acceso transparente a la información financiera en tiempo real.

1.6. Arquitectura del sistema

La arquitectura se fundamenta en el **Modelo de Vistas 4+1**, permitiendo una comprensión integral del software desde diversas perspectivas:

- **Vista de Escenario:** Define la funcionalidad mediante casos de uso.
- **Vista Lógica:** Describe la estructura de clases y el modelo de datos.
- **Vista de Proceso:** Modela el comportamiento y flujo del sistema.
- **Vista de Desarrollo:** Organiza los componentes y paquetes de software.
- **Vista Física:** Detalla la topología del hardware y despliegue en la nube.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema se concibe como una solución omnicanal que permite a los residentes visualizar sus deudas, realizar pagos electrónicos y gestionar requerimientos desde una interfaz única (con funciones como "Pagar", "Instalaciones" y "Mensajes"). Para la administración, actúa como un panel de control para la generación de informes, supervisión de personal y seguimiento de morosidades.

2.2. Objetivos del sistema

- **Automatizar** el ciclo de facturación y cobranza de gastos comunes.
- **Garantizar la transparencia** financiera mediante reportes descargables para los copropietarios.
- **Centralizar la comunicación** y la gestión de reclamos para mejorar la trazabilidad de las respuestas.
- **Reducir la morosidad** mediante recordatorios automáticos y sistemas de pago integrados.
- **Elevar los estándares de seguridad** del recinto mediante la implementación de un módulo de gestión de visitas, permitiendo un control riguroso y preventivo de los accesos al edificio.

2.3. Requerimientos funcionales

- **RF-01 Registro y gestión de residentes**

El sistema debe permitir al administrador registrar, modificar y dar de baja los datos de todos los residentes, diferenciando entre propietarios y arrendatarios. Cada residente debe quedar estrictamente vinculado a uno de los 160 departamentos del edificio.

- **RF-02 Registro y gestión de personal**

El sistema debe permitir al administrador registrar, modificar y dar de baja los datos del personal contratado por el edificio, asignándoles un rol y credenciales de acceso a la plataforma.

- **RF-03 Gestión de permisos y roles de usuario**

El sistema debe permitir al administrador gestionar el nivel de acceso de cada cuenta, asignando o modificando los roles predefinidos (Administrador, Propietario, Arrendatario, Conserje, Mantenimiento) para controlar qué módulos de la plataforma puede ver o editar cada persona.

- **RF-04 consultar los datos de residente**

El sistema debe contar con un buscador general que permita al administrador y a los conserjes consultar rápidamente la información de contacto y la unidad asociada de cualquier residente activo en el edificio.

- **RF-05 Registro y gestionar de departamentos**

El sistema debe permitir al administrador registrar e identificar cada una de las 160 unidades (departamentos) que componen el edificio, estableciendo sus características físicas y parámetros de cobro.

- **RF-06 Registro y gestión de contratos de arriendo**

El sistema debe permitir registrar formalmente el vínculo de arriendo entre un inquilino y un departamento, adjuntando el documento de respaldo y definiendo quién es el responsable legal de pagar los gastos comunes.

- **RF-07 Registro fotográfico de condiciones del dpto.**

El sistema debe permitir subir y almacenar evidencia fotográfica del estado físico del departamento (inventario visual) al momento de iniciar un nuevo contrato de arriendo o tras realizar una reparación importante.

- **RF-08 Registro y configuración de fecha de cobro**

El sistema debe permitir al administrador definir y modificar el ciclo de facturación mensual, estableciendo la fecha exacta de emisión de las cuotas y la fecha límite de pago para los residentes.

- **RF-09 Registro y validación de pagos**
El sistema debe permitir registrar los pagos de gastos comunes realizados por los residentes, ya sea validando comprobantes de transferencia manuales o registrando transacciones automáticas, para actualizar el saldo de la deuda en tiempo real.
- **RF-10 Consultar fecha de vencimiento de próxima cuota**
El sistema debe mostrar al residente de forma clara y accesible la fecha límite para el pago de su próxima cuota de gastos comunes, calculando los días restantes para prevenir atrasos.
- **RF-11 Cálculo automático de GGCC mensuales**
El sistema debe automatizar el cálculo de la cuota mensual de gastos comunes para cada departamento, sumando los egresos totales del periodo (luz, agua, gas, remuneraciones, mantenciones) y prorrateándolos según el porcentaje de participación de cada unidad.
- **RF-12 Integración con plataforma de pago online**
El sistema debe integrar una plataforma de pagos electrónicos (Webpay, Mercado Pago, Khipu, etc.) para permitir a los residentes pagar sus cuotas de gastos comunes directamente desde la plataforma, utilizando tarjetas de crédito o débito.
- **RF-13 Consulta de estado de cuenta del dpto.**
El sistema debe generar y mostrar un registro histórico detallado (cartola) de todos los movimientos financieros de un departamento, incluyendo cargos (emisión de gastos comunes, multas) y abonos (pagos realizados), reflejando el saldo actualizado.
- **RF-14 Identificación y panel de dptos. Morosos**
El sistema debe identificar automáticamente y consolidar en un panel de control a todos los departamentos que mantengan cuotas de ggcc vencidas, permitiendo a la administración visualizar el tiempo de atraso y gestionar el cobro.
- **RF-15 Ingreso de solicitudes y ticket de mantención**
El sistema debe permitir a los residentes y conserjes generar "tickets" formales para reportar problemas de infraestructura en áreas comunes o solicitar reparaciones, centralizando la comunicación y evitando solicitudes informales.
- **RF-16 Actualización y seguimiento de tareas de mantención**
El sistema debe permitir al administrador asignar solicitudes a los encargados de mantención, y permitir a estos últimos actualizar el estado de avance de la tarea, dejando un registro de la solución.

- **RF-17 Registro de incidentes y novedades del edificio**

El sistema debe permitir a los conserjes registrar sucesos relevantes ocurridos durante su turno (robos, ruidos molestos, fallas de seguridad o emergencias), creando una bitácora digital de libre consulta para la administración.

- **RF-18 Registro de visitas y recepción de encomiendas**

El sistema debe permitir a los conserjes registrar el ingreso de visitas y la recepción de paquetes o correspondencia, notificando automáticamente al residente correspondiente para su retiro o autorización.

- **RF-19 Publicación de avisos y circulares informativas**

El sistema debe permitir al administrador redactar y difundir comunicados oficiales, reglamentos o avisos de utilidad pública (cortes de agua, mantención de ascensores, etc.) de forma masiva a todos los residentes.

- **RF-20 Reserva y gestión de espacios comunes**

El sistema debe permitir a los residentes visualizar la disponibilidad y solicitar la reserva de áreas comunes (Quincho, sala de Eventos, etc), gestionando los horarios y el cargo automático del costo de arriendo.

- **RF-21 Historial y bitácora de actividades**

El sistema debe registrar automáticamente cada acción crítica realizada por los usuarios (creación, modificación o eliminación de datos), guardando un historial inmodificable para auditorías posteriores.

- **RF-22 Gestión de respaldo de información**

El sistema debe realizar copias de seguridad automáticas de toda la base de datos y archivos adjuntos (fotos/PDFs) para garantizar la recuperación de la información ante fallas técnicas.

- **RF-23 Búsqueda avanzada y filtros**

El sistema debe proporcionar una herramienta de búsqueda que permita localizar información cruzada (residentes, pagos, unidades, tickets) mediante el uso de múltiples filtros simultáneos.

2.4. Requerimientos no funcionales

- **RNF-01 Seguridad y protección de datos**

El sistema debe garantizar que la información sensible de los residentes esté protegida contra accesos no autorizados.

- **RNF-02 Trazabilidad de acciones**

Capacidad de reconstruir eventos pasados dentro del sistema.

- **RNF-03 Disponibilidad del sistema**

El sistema debe garantizar que el acceso a la plataforma esté operativo la mayor parte del tiempo, asegurando que la gestión de conserjería no se vea interrumpida por fallas técnicas.

- **RNF-04 Recuperabilidad ante fallos**

Capacidad del sistema para restaurar datos en caso de una falla catastrófica del servidor.

- **RNF-05 Usabilidad y diseño responsivo**

La interfaz debe ser intuitiva y adaptarse a computadores, tablets y smartphones.

- **RNF-06 Localización**

El sistema debe estar adaptado al contexto local de Chile.

- **RNF-07 Accesibilidad web**

El sistema debe ser usable por personas con capacidades visuales reducidas.

- **RNF-08 Tiempo de respuesta y rendimiento**

El sistema debe procesar las solicitudes de manera ágil para asegurar la eficiencia en la atención y cobros.

- **RNF-09 Capacidad de almacenamiento**

El sistema debe proveer espacio suficiente para los archivos adjuntos.

- **RNF-10 Escalabilidad de carga**

El sistema debe ser capaz de manejar un aumento en el número de departamentos o residentes sin pérdida de rendimiento.

- **RNF-11 Compatibilidad de navegadores**

La plataforma debe ser accesible desde los navegadores web más utilizados en el mercado chileno.

- **RNF-12 Integridad y consistencia de datos**

El sistema debe garantizar que los cálculos financieros sean exactos y que no se pierda información durante transacciones interrumpidas.

- **RNF-13 Facilidad de mantenimiento**

El software debe estar construido de forma que sea fácil corregir errores o agregar mejoras en el futuro.

- **RNF-14 Manejo de errores**

El sistema no debe cerrarse inesperadamente ante entradas de datos erróneas del usuario.

2.5. Supuestos y dependencias

- **Supuesto:**

- **Conectividad de Residentes:** Se asume que el 100% de los copropietarios y arrendatarios poseen acceso a dispositivos móviles con sistemas operativos vigentes (iOS/Android) o computadores con navegadores web actualizados para interactuar con la plataforma.
- **Disponibilidad de Datos Históricos:** Se asume que la administración actual posee registros (aunque sean manuales) de las deudas históricas, datos de contacto de emergencia y tipos de residencia para la carga inicial del sistema.
- **Capacidad del Personal:** Se asume que los 2 conserjes y las 3 personas de mantención recibirán una capacitación básica para el uso del módulo de solicitudes y seguridad, integrándolo en su rutina laboral diaria.
- **Integridad de la Red:** Se asume que el edificio cuenta con una infraestructura de red estable en áreas comunes para permitir el funcionamiento de funciones en tiempo real, como la apertura remota de portones.

- **Dependencia:**

- **Plataformas de Pago Externas:** El correcto funcionamiento del módulo de "Pagos" depende de la integración exitosa y la estabilidad de APIs de terceros (como Transbank, Webpay o Flow) para el procesamiento de transacciones financieras.
- **Marco Legal y Normativo:** La implementación del sistema de cobros y multas depende de que las reglas de cálculo estén alineadas con el Reglamento de Copropiedad del edificio y la legislación vigente sobre gastos comunes.
- **Sincronización de Base de Datos:** La funcionalidad de visualización de deudas en tiempo real depende de la sincronización constante entre el backend del sistema y el motor de base de datos SQL que gestionará los saldos.
- **Hardware de Seguridad Existente:** La función de "Abrir Portón" depende de la compatibilidad técnica entre el software y los sistemas físicos de automatización (PLCs o controladores de acceso) ya instalados en el edificio.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Para el sistema Vecinos Conectados se ha definido un estilo monolítico con una arquitectura organizada en capas. Esto significa que todos los componentes del software se despliegan como una única unidad lógica, pero su código interno está separado en capas de Presentación, Lógica de Negocio y Acceso a Datos para mantener el orden y la mantenibilidad.

3.2. Justificación del estilo según el contexto del sistema

Se selecciona este estilo debido a que la complejidad del proyecto (gestión de 160 departamentos) no requiere la sobrecarga de una arquitectura distribuida. Un monolito facilita el desarrollo inicial, simplifica las pruebas de integración y permite una comunicación directa entre los módulos de pagos, visitas y administración sin latencia de red.

3.3. Patrones de diseño aplicados

El patrón de diseño aplicado es MVC.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

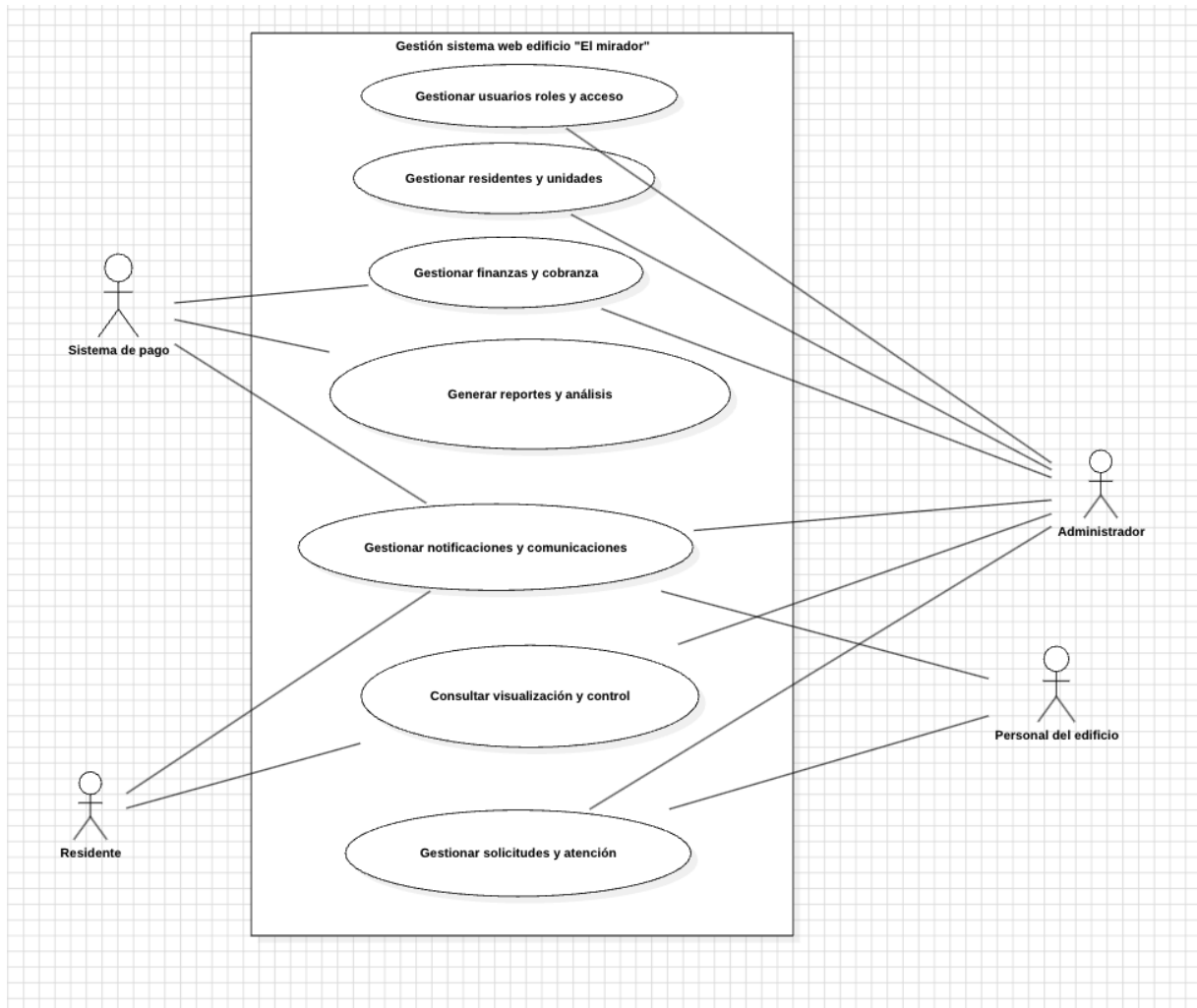
4.1.1. Propósito

El propósito de esta vista es describir las funcionalidades del sistema desde la perspectiva de los usuarios (Residentes, Administradores y Personal del edificio). Su objetivo es validar que la arquitectura responda a los requerimientos del negocio, como la gestión de pagos y la seguridad del edificio, sirviendo de base para el diseño de las demás vistas técnicas.

4.1.2. Actores

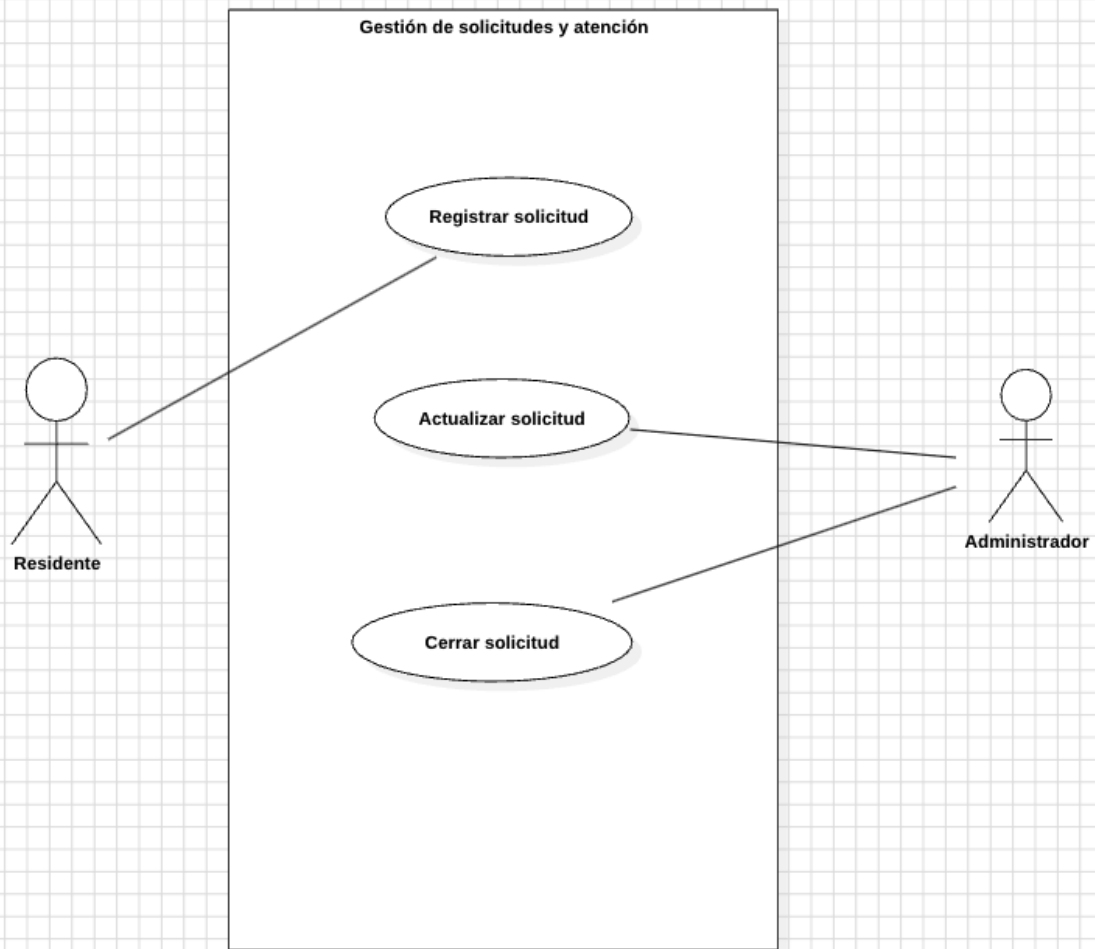
- **Administrador:** Super usuario con permisos totales para gestionar residentes, calcular gastos comunes, asignar roles y generar informes de morosidad
- **Residentes:** Propietarios y arrendatarios que utilizan la plataforma para pagar cuotas, enviar solicitudes de mantenimiento y autorizar visitas.
- **Personal del edificio:** Incluye a conserjes y personal de mantención, quienes interactúan con el sistema para registrar ingresos de visitas y gestionar las solicitudes de reparación de las instalaciones.
- **Sistema de pagos externos:** Entidad externa que procesa las transacciones financieras y confirma el pago de las cuotas al sistema.

4.1.3. Diagrama general de casos de uso



El diagrama de caso de uso general define la forma en que se comporta el software a través de las interacciones entre los actores y los servicios del sistema.

4.1.4. Diagrama de casos de uso específicos



El caso de uso específico es la gestión de solicitudes de mantenimiento donde los residentes pueden ingresar solicitudes y el personal del edificio recibe las solicitudes, también el personal puede actualizar las solicitudes. El caso de uso cerrar solicitud es una desactivación lógica del registro en la base de datos. El administrador también puede actualizar y desactivar solicitudes.

4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-01	Actualizar estado de tareas	Personal
CU-02	Solicitud de mantenimiento	Residentes, Personal
CU-03	Realizar pago	Residentes, Sistema de pago
CU-04	Consultar estado de cuenta	Residentes
CU-05	Gestión de notificaciones	Residentes, Administrador
CU-06	Gestión de reclamos y solicitudes	Residentes, Administrador
CU-07	Gestionar usuarios y permisos	Administrador
CU-08	Generar reportes	Administrador
CU-09	Gestionar morosidad	Administrador
CU-10	Administrar pagos	Administrador
CU-11	Registro de solicitudes y mantenimiento	Administrador, Personal del edificio, Residentes
CU-12	Gestión de solicitudes de quejas	Administrador, personal del edificio y residentes
CU-13	Actualizar solicitud	Personal del edificio, Administrador
CU-14	Cerrar solicitud	Personal del edificio, Administrador
CU-15	Registrar solicitud	Residentes

4.1.6. Especificación de casos de uso

Caso de Uso	Registro de solicitudes y mantenimiento	Identificador: CU-11
Actores	Residente (Iniciador), Personal del edificio (Ejecutor) y Administrador	
Tipo	Primario	
Referencias	RF-15 registro de solicitudes de mantenimiento	
Precondición	El usuario debe estar autenticado en la plataforma y pertenecer a una unidad activa	
Postcondición	Se genera un ticket con numero de seguimiento y se notifica al personal responsable	
Descripción	Permite a los residentes fallas en infraestructura o solicitar reparaciones técnicas	
Resumen	El residente crea una solicitud, el administrador la asigna y el personal del edificio actualiza el avance hasta el cierre.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Residente	Ingresa al módulo "Solicitudes" y selecciona "crear nuevo ticket"
2	Residente	Describe el problema (ej: filtración) y adjunta evidencia fotográfica si aplica
3	Sistema	Registra solicitud y asigna un estado inicial de "Pendiente"
4	Administrador	Revisa el ticket y lo asigna al personal del edificio
5	Personal del edificio	Actualiza el estado a "En proceso" y finalmente a "Resuelto" tras la reparación.

CURSO ALTERNATIVO

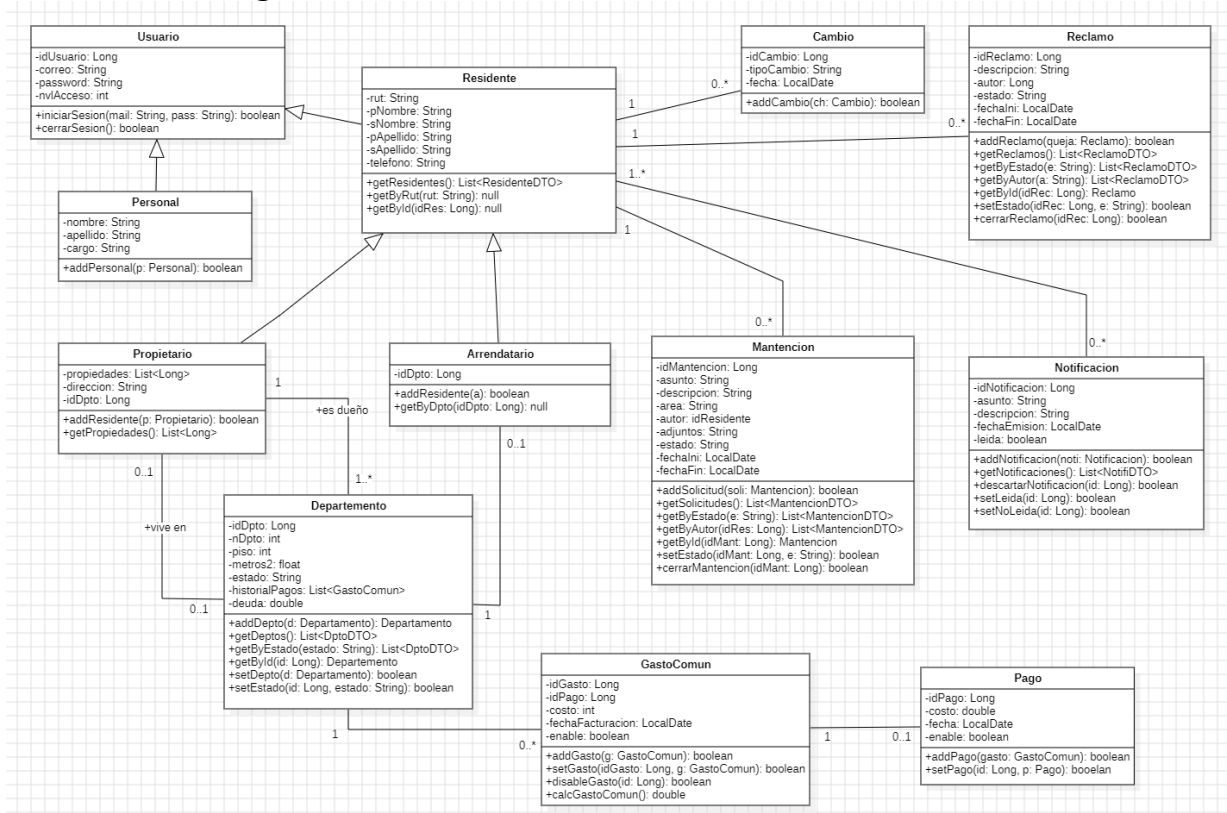
Nro.	Descripción de acciones alternas
1a	Solicitud duplicada: Si ya existe un reporte similar en la misma área común, el sistema sugiere unirse al ticket existente
2a	Rechazo de solicitud: Si la reparación no corresponde a la administración, el administrador cierra el ticket indicando el motivo.
3a	Falta de insumos: El personal marca el ticket como "Pausado" especificando la falta de repuestos o materiales.

4.2. VISTA LÓGICA

4.2.1. Propósito

El propósito de esta vista es modelar la estructura lógica interna del sistema mediante clases y relaciones, permitiendo representar las entidades principales del dominio y su interacción dentro de la solución propuesta.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

El diagrama de clases representa la estructura lógica del sistema Vecinos Conectados, modelando las entidades principales, sus atributos y relaciones para soportar la gestión operativa del edificio.

La clase **Usuario** actúa como entidad base del sistema, concentrando atributos comunes como identificación, credenciales y datos de contacto. A partir de esta clase se especializan distintos perfiles mediante herencia, tales como **Administrador**, **Residente** y **Personal**, permitiendo diferenciar permisos y responsabilidades dentro de la plataforma.

La clase **Departamento** representa cada una de las unidades habitacionales del edificio y mantiene asociación con residentes, contratos y registros financieros. Esta relación permite identificar responsables de pago, historial de ocupación y características de cada unidad.

La clase **Pago** modela las transacciones de gastos comunes realizadas por los residentes, almacenando información como monto, fecha, estado y método de pago. Se relaciona directamente con el residente y el departamento correspondiente, permitiendo trazabilidad financiera y actualización de saldos.

La clase **Solicitud Mantenimiento** representa los tickets generados para reportar incidencias o requerimientos técnicos. Incluye atributos como descripción, prioridad, estado y fecha de creación, además de relaciones con residentes, administrador y personal encargado de resolver la incidencia.

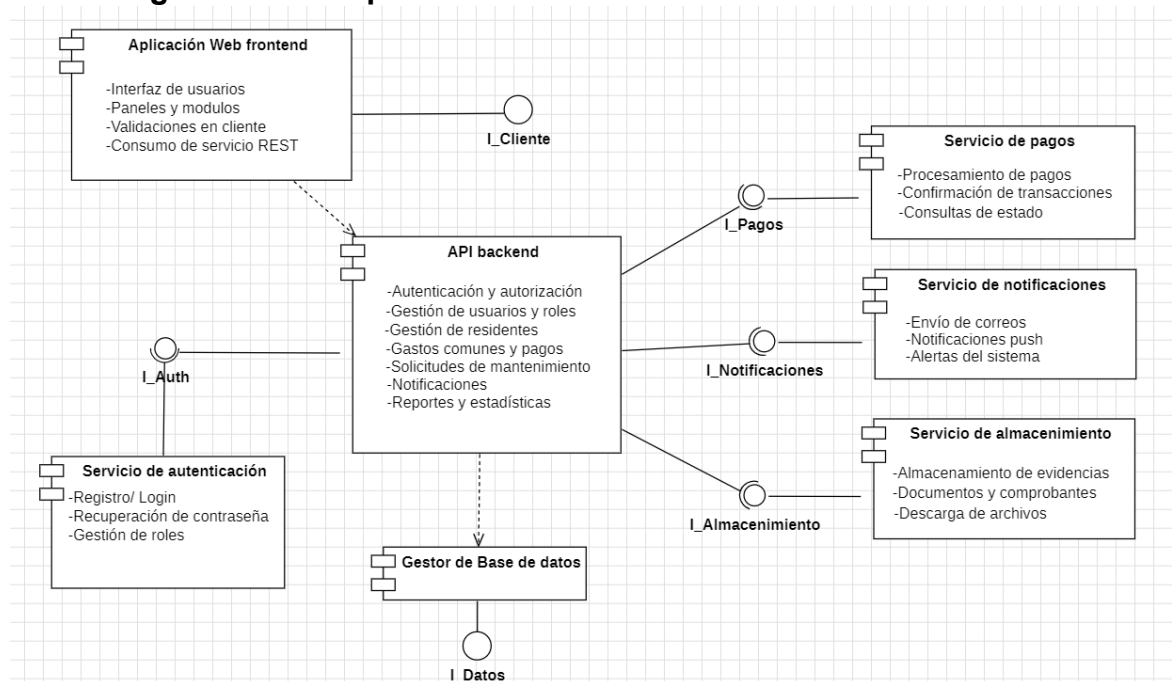
Finalmente, el modelo incorpora relaciones de asociación, herencia y dependencia que permiten estructurar el sistema de forma modular, facilitando la gestión de pagos, mantenimiento, usuarios y control operativo del edificio.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

El propósito de esta vista es describir la organización física de los componentes de software que integran el sistema "Vecinos Conectados". Su objetivo es detallar cómo se estructura el código fuente bajo un esquema monolítico, definiendo la jerarquía de paquetes basada en el patrón MVC y la gestión de dependencias externas para asegurar un desarrollo ordenado y un mantenimiento eficiente de las funcionalidades del edificio.

4.3.2. Diagrama de componente

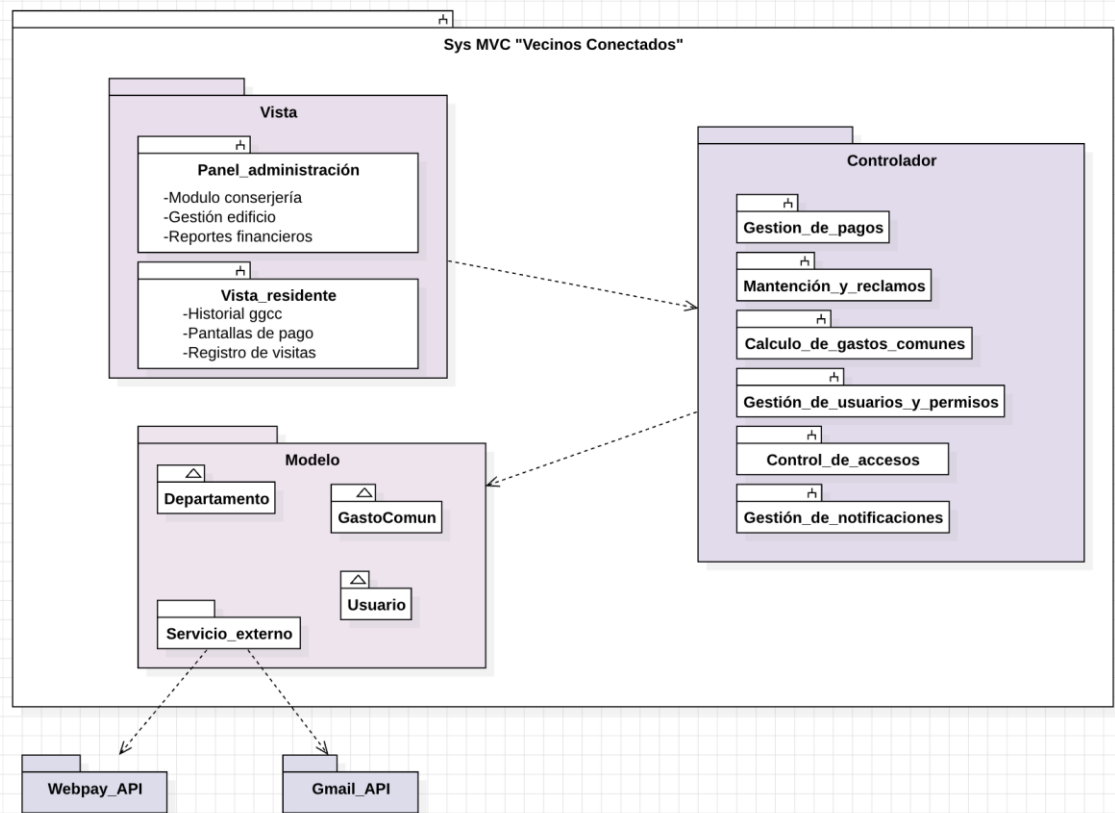


4.3.3. Descripción diagrama de componente

El diagrama detalla la organización física y lógica de los componentes de software que conforman el sistema Vecinos Conectados, así como sus interfaces de comunicación e integración.

- **Aplicación Web (Frontend):** componente encargado de la interfaz gráfica utilizada por residentes, administración y personal del edificio. Gestiona la interacción del usuario mediante vistas dinámicas y consume los servicios del servidor a través de peticiones REST.
- **API Backend (Monolito):** constituye el núcleo principal del sistema y centraliza la lógica de negocio. Procesa la autenticación de usuarios, la gestión de residentes y departamentos, el cálculo de gastos comunes, el procesamiento de pagos y la generación de reportes.
- **Servicios internos especializados:** módulos organizados dentro del backend que resuelven funcionalidades específicas del sistema:
- **Servicio de autenticación:** controla el acceso seguro a la plataforma, validación de credenciales y asignación de roles y permisos.
- **Servicio de pagos:** administra el flujo de transacciones financieras e integración con plataformas externas como Webpay o Transbank.
- **Servicio de notificaciones:** gestiona el envío automático de alertas, recordatorios, avisos del sistema y correos electrónicos a residentes.
- **Servicio de almacenamiento:** permite la gestión y persistencia de archivos físicos, tales como evidencias fotográficas, documentos y comprobantes de pago.
- **Gestor de Base de Datos:** componente relacional responsable del almacenamiento persistente e íntegro de la información del sistema, incluyendo usuarios, pagos, solicitudes y registros históricos.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

El diagrama de paquetes describe la organización interna del monolito "Vecinos Conectados", estructurado bajo el patrón MVC para asegurar una gestión modular de los requerimientos del edificio

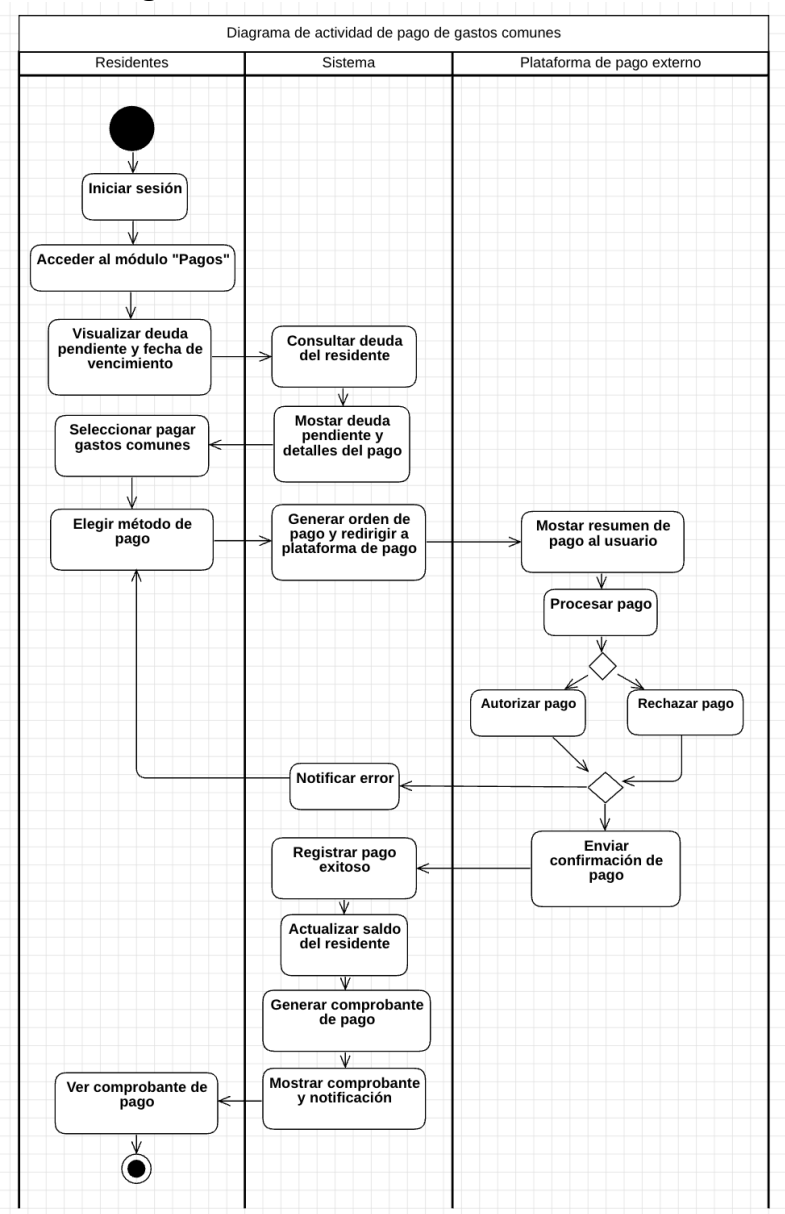
- **Vista:** Contiene las interfaces de usuario que permiten la interacción de residentes, administración y personal.
- **Controlador:** Actúa como el núcleo lógico que procesa las solicitudes de la vista y coordina las respuestas mediante la lógica de negocio.
- **Modelo:** Gestiona la persistencia de datos y la integridad de la información del edificio.
- **Servicio externo:** Submódulo del modelo que centraliza la comunicación con plataformas fuera del sistema.
- **Webpay API (Externo):** Representa la pasarela de pagos. La dependencia indica que el monolito delega el procesamiento financiero a este servicio para garantizar transacciones seguras.

4.4. VISTA DE PROCESOS

4.4.1. Propósito

El propósito de este diagrama es modelar el flujo de control y el comportamiento dinámico del sistema ante procesos críticos del edificio. Busca visualizar la secuencia de actividades y decisiones lógicas que ocurren, por ejemplo, durante el flujo de un pago de gastos comunes o la validación de una visita, permitiendo identificar cómo interactúan los módulos del monolito para completar una tarea de forma eficiente.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El diagrama modela el comportamiento dinámico y el flujo de control del proceso de recaudación de gastos comunes correspondiente al caso de uso CU-03: Realizar pago. La lógica del proceso se distribuye en cuatro líneas de responsabilidad que interactúan de la siguiente manera:

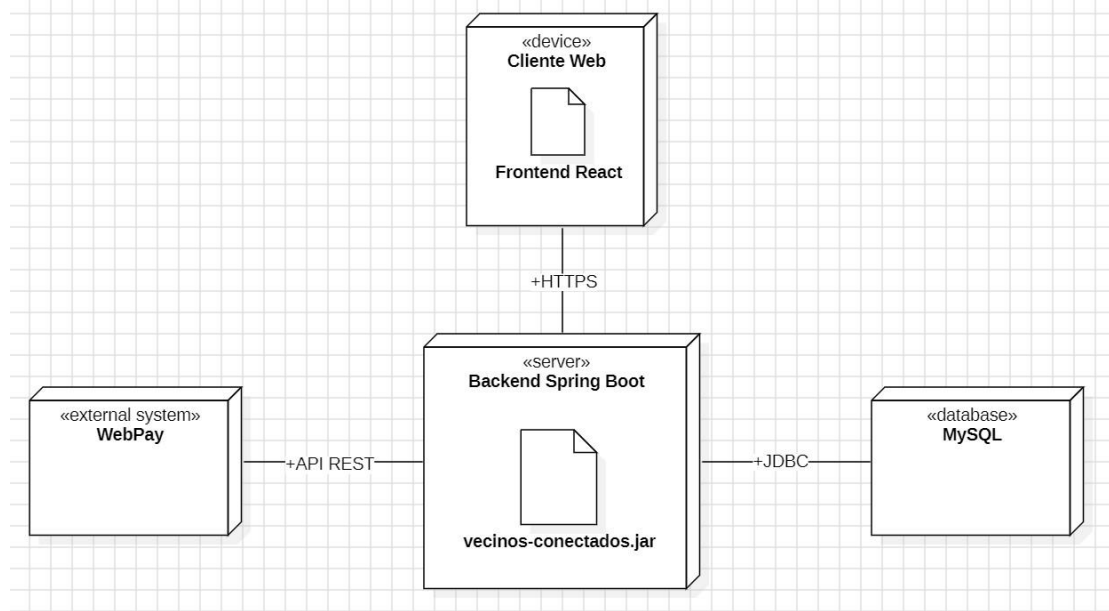
- **Residente:** inicia el flujo autenticándose en la plataforma y accediendo al módulo de pagos. Luego visualiza su deuda pendiente, selecciona la opción de pago y elige el método correspondiente. Una vez completada exitosamente la transacción, puede visualizar y descargar su comprobante digital.
- **Sistema:** actúa como núcleo central de control del proceso. Se encarga de consultar y mostrar la deuda vigente del residente, generar la orden de pago y coordinar la comunicación entre la plataforma y los servicios externos. Tras recibir la confirmación de pago, registra la transacción, actualiza el saldo del residente y genera el comprobante.
- **Plataforma de pago externa:** recibe la redirección desde el sistema, presenta al usuario el resumen de la operación y procesa la transacción financiera. Posteriormente, envía la respuesta del pago al sistema para continuar con el flujo.
- **Procesador de pagos:** corresponde al servicio bancario encargado de validar y autorizar la operación financiera. Existen dos posibles resultados: autorización del pago, permitiendo finalizar exitosamente la transacción, o rechazo del pago, lo que genera una notificación de error y permite al residente volver a seleccionar un método de pago.

4.5. VISTA FÍSICA

4.5.1. Propósito

El propósito de este diagrama es representar la arquitectura física y la distribución de los componentes del software en el hardware. Detalla la topología de red, los servidores donde se aloja el monolito, el motor de base de datos SQL y la conectividad necesaria para que los 160 departamentos y la administración accedan a la plataforma desde sus dispositivos móviles o computadores.

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

El diagrama representa la arquitectura física y la distribución de los componentes de software en la infraestructura de hardware, adoptando un esquema centralizado apto para una arquitectura monolítica:

- **Dispositivos Cliente (Nodos de Acceso):** Representan los computadores y smartphones utilizados por los residentes, la administración y el personal del edificio. El acceso se realiza de forma remota a través de navegadores web mediante el protocolo seguro HTTPS.
- **Servidor de Aplicaciones (Servidor Web):** Nodo central que aloja el monolito ejecutable del sistema. Este servidor procesa todas las solicitudes provenientes de las interfaces (Capa de Presentación) y ejecuta la lógica de negocio (Capa de Control) de manera unificada.
- **Servidor de Base de Datos:** Nodo independiente que contiene el motor de base de datos SQL. Se comunica directamente con el servidor de aplicaciones dentro de la red interna para garantizar una baja latencia en el almacenamiento y consulta de la información de los 160 departamentos.
- **Servicio Externo (Webpay API):** Representa la infraestructura externa de la pasarela de pagos. La conexión se realiza a través de Internet desde el servidor de aplicaciones para procesar de manera segura las transacciones financieras.

5. REQUISITOS DE CALIDAD

5.1. Propósito

El propósito es definir y cuantificar los atributos de calidad (requerimientos no funcionales) que gobiernan el comportamiento del sistema "Vecinos Conectados". El establecimiento de estas métricas asegura que el diseño monolítico en capas no solo sea funcional, sino que además sea seguro, eficiente, escalable y mantenible en el tiempo para la comunidad de 160 departamentos del edificio "El Mirador".

5.2. Atributos de calidad

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN
Seguridad y Confidencialidad	Garantía de protección de datos sensibles financieros y personales contra accesos no autorizados	Crítico para el cumplimiento legal: Protege los datos bancarios en el proceso de pago e implementa cifrado en el módulo de denuncias para cumplir con la Ley Karin. Se refleja en el <i>Diagrama de Despliegue</i> al aislar el servidor MySQL en una red interna y forzar tráfico cifrado bajo HTTPS
Mantenibilidad	Capacidad del software para ser modificado, corregido o mejorado con el menor esfuerzo y riesgo posible	Justificación por el diseño: Al adoptar una arquitectura monolítica con el patrón MVC, cualquier cambio en las interfaces del frontend no corrompe las reglas del negocio ni el modelo de datos. Se evidencia en la alta cohesión del <i>Diagrama de Paquetes</i>
Rendimiento y Tiempo de Respuesta	Capacidad del sistema para procesar transacciones y desplegar información dentro de límites de tiempo aceptables	Crítico para la conserjería: El registro de visitas y recepción de encomiendas debe ser inmediato para evitar aglomeraciones en el acceso peatonal del edificio. Se optimiza en el <i>Diagrama de Despliegue</i> eliminando la latencia de red mediante la centralización en un único Servidor de Aplicaciones Spring Boot
Disponibilidad	Fracción de tiempo en la que el sistema se encuentra operativo y accesible para realizar sus funciones.	Crítico para la comunidad: Los conserjes operan en turnos 24/7 y la gestión de seguridad no puede interrumpirse. Se sustenta mediante el <i>Diagrama de Componentes</i> , aislando los servicios especializados para evitar caídas globales del sistema.

5.3. Reglas y criterios de evaluación de calidad

Para auditar y garantizar el cumplimiento de los estándares propuestos, se establecen las siguientes métricas cuantitativas y metodologías de evaluación:

1. Seguridad y Confidencialidad (Cumplimiento Ley Karin)

- Criterio: El 100% de las conexiones externas deben viajar cifradas bajo el protocolo HTTPS. Las contraseñas en la clase Usuario deben estar encriptadas en la base de datos mediante algoritmos de hash (ej. BScript).
- Evaluación: Pruebas de penetración web (Penetration Testing) e inspección automatizada de código para mitigar riesgos de inyección SQL.

2. Mantenibilidad (Arquitectura en Capas - MVC)

- Criterio: Mantener un bajo acoplamiento y alta cohesión. Cualquier modificación en un servicio especializado (ej. Servicio de notificaciones) debe tener un 0% de impacto en las líneas de código del controlador financiero principal.
- Evaluación: Análisis estático de código mediante SonarQube, exigiendo una calificación de mantenibilidad nivel "A".

3. Rendimiento y Tiempo de Respuesta

- Criterio: Las consultas de saldos, paneles de morosidad y registros de visitas en conserjería deben desplegar la información en un tiempo menor a 2.0 segundos bajo condiciones normales.
- Evaluación: Pruebas de estrés y carga automatizadas con Apache JMeter, simulando la concurrencia máxima del edificio el día de vencimiento de los gastos comunes.

4. Disponibilidad del Sistema (Operación 24/7)

- Criterio: Garantizar un Acuerdo de Nivel de Servicio (SLA) de disponibilidad del 99.5% mensual, limitando el tiempo de inactividad técnica imprevista a un máximo de 3.6 horas al mes.
- Evaluación: Monitoreo continuo de infraestructura mediante sondas automatizadas (ej. Uptime Robot) con alertas inmediatas ante pérdidas de conectividad.

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

El propósito es justificar cómo los principios fundamentales de la ingeniería de software guiaron la construcción de las interfaces interactivas en Figma. La aplicación de estos principios asegura que el prototipo no sea solo una maqueta visual, sino una simulación fiel de la arquitectura modular y limpia definida para el sistema.

6.2. Principios de diseño

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Modularidad	Dividir el sistema en componentes visuales e interfaces independientes para facilitar la comprensión y reutilización.	Estructura por Componentes UI: Las interfaces de inicio, Login y del requerimiento asignado se diseñaron utilizando <i>Componentes</i> y <i>Variantes</i> en Figma, permitiendo que cualquier cambio en la barra de navegación o botones se replique automáticamente en todas las vistas del prototipo.
Encapsulación	Ocultar los detalles internos y operativos de los componentes, mostrando exclusivamente lo necesario para su uso inmediato	Flujos de Entrada Controlados: En la interfaz de Login, los detalles de verificación técnica quedan ocultos tras el botón interactivo. La interfaz solo expone los campos controlados de texto necesarios para que el usuario experimente la autenticación segura
Abstracción	Simplificar la complejidad del sistema del mundo real enfocándose en los elementos y flujos esenciales para el usuario	Simulación del Negocio: Para las 160 unidades del edificio, se abstraen flujos complejos de bases de datos relacionales en interacciones visuales directas (ej. el botón "Pagar con Webpay" simplifica la pasarela transaccional externa en un solo flujo interactivo de clics).
Cohesión	Propiedad que busca que cada pantalla, módulo o componente especializado resuelva una única tarea del negocio	Alta Cohesión de Pantallas: La interfaz del requerimiento asignado (ej. Gestión de Pagos) se enfoca exclusivamente en mostrar saldos y el historial financiero, eliminando elementos distractores de conserjería o mantenimiento
Acoplamiento	Grado de interdependencia o conexión que posee una interfaz o un módulo respecto a otros dentro del sistema	Bajo Acoplamiento por Conexiones: Los flujos dinámicos interconectan las vistas utilizando conectores lógicos independientes. Si se modifica el diseño del portal de reclamos, el flujo del Login no sufre alteraciones en su cableado interactivo.

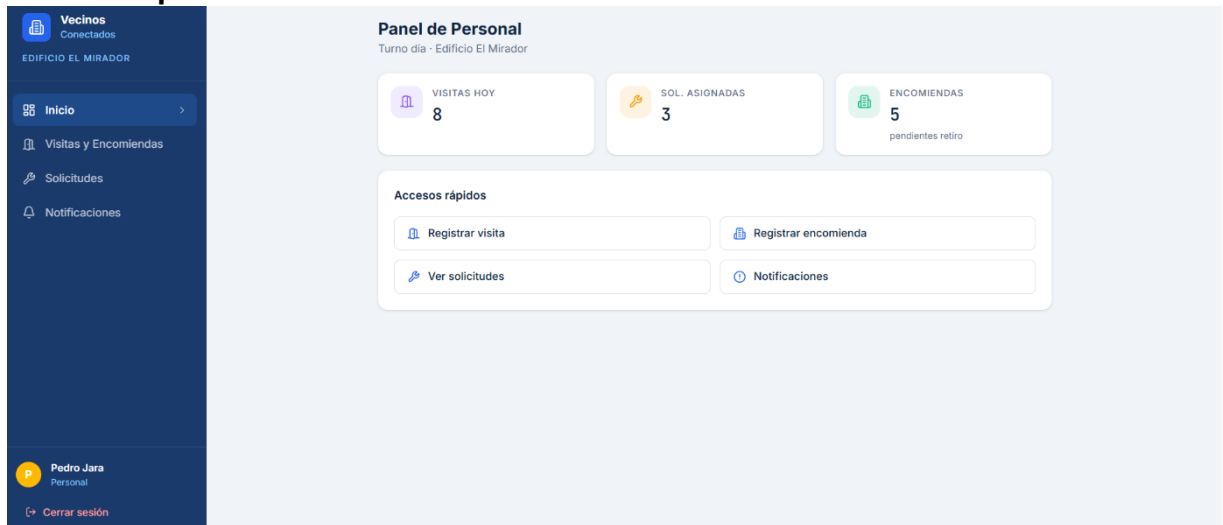
7. PROTOTIPO

7.1. Propósito

El propósito del prototipado es construir una versión interactiva de alta fidelidad que simule el comportamiento real del sistema "Vecinos Conectados" ante los escenarios críticos del edificio "El Mirador". Esto permite validar de forma empírica la satisfacción del usuario y refinar los flujos lógicos antes de iniciar la codificación final.

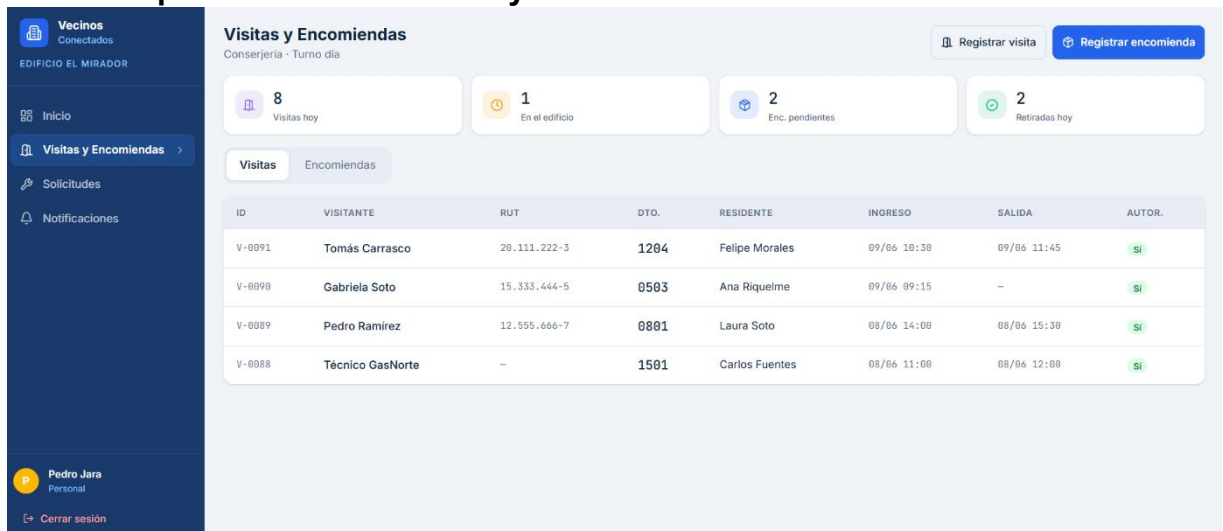
7.2. Mockups (imágenes con una breve descripción)

Mockup 1: Panel de Inicio - Personal del Edificio



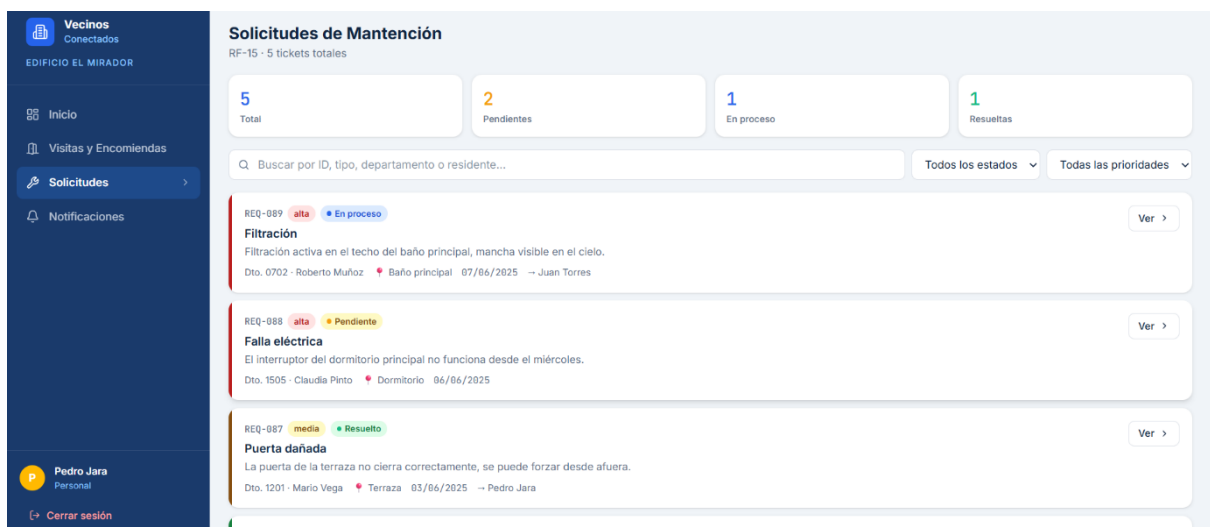
Esta interfaz corresponde al panel principal del personal del edificio una vez iniciada la sesión. En ella se presenta un resumen de las actividades del día, mostrando indicadores como el número de visitas registradas, solicitudes asignadas y encomiendas pendientes de entrega. Además, incorpora accesos rápidos a las funcionalidades más utilizadas, como registrar visitas, registrar encomiendas, consultar solicitudes y revisar notificaciones, facilitando el trabajo diario del personal de conserjería.

Mockup 2: Gestión de Visitas y Encomiendas - Personal del Edificio



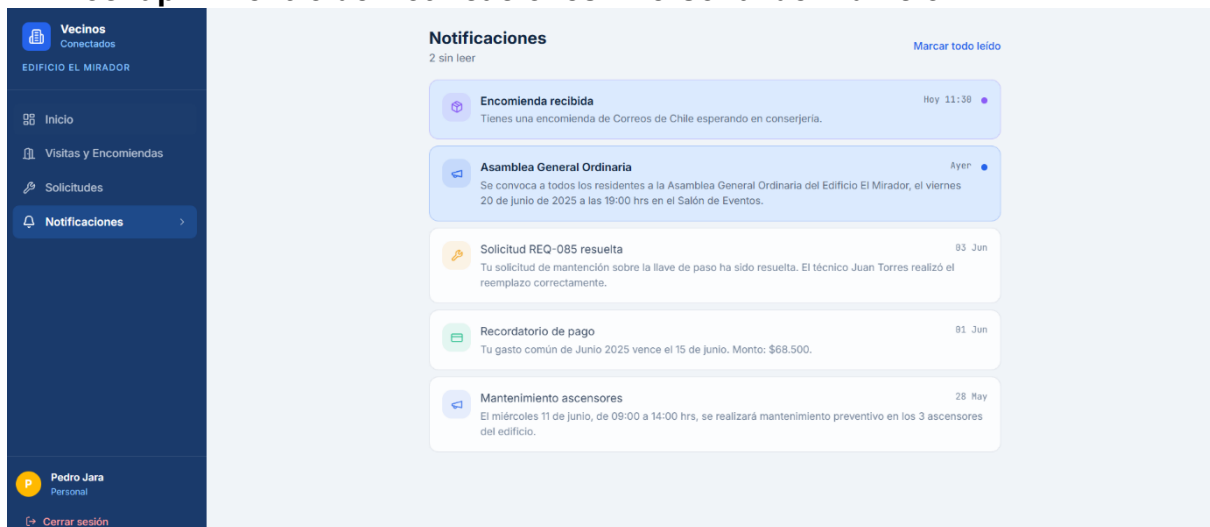
Esta pantalla permite al personal registrar y administrar el ingreso de visitas y la recepción de encomiendas del edificio. Se visualizan indicadores con el total de visitas del día, personas actualmente dentro del edificio, encomiendas pendientes y entregas realizadas. Asimismo, se incluye una tabla con el historial de registros, permitiendo consultar información como visitante, residente asociado, departamento, horario de ingreso y salida, fortaleciendo el control de acceso y la trazabilidad de los registros.

Mockup 3: Gestión de Solicitudes de Mantenimiento - Personal del Edificio



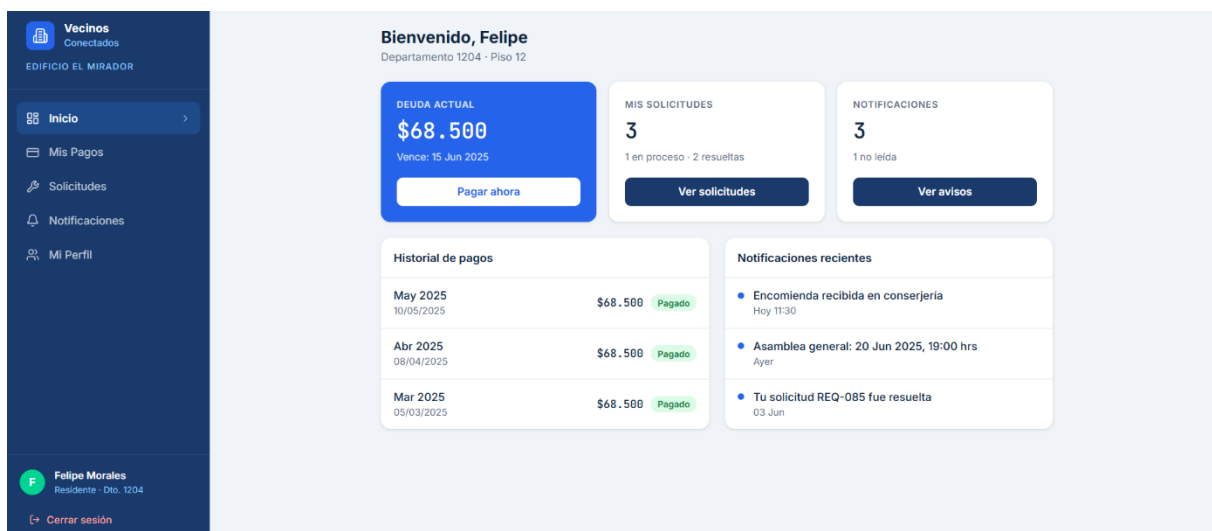
Esta interfaz corresponde al módulo de solicitudes de mantenimiento, donde el personal del edificio puede visualizar los tickets registrados por los residentes. La pantalla presenta indicadores del total de solicitudes y su estado (pendientes, en proceso y resueltas), además de herramientas de búsqueda y filtrado por prioridad y estado. Cada solicitud muestra información relevante como descripción del problema, ubicación, residente solicitante y encargado asignado, permitiendo realizar un adecuado seguimiento de las tareas de mantenimiento.

Mockup 4: Centro de Notificaciones - Personal del Edificio



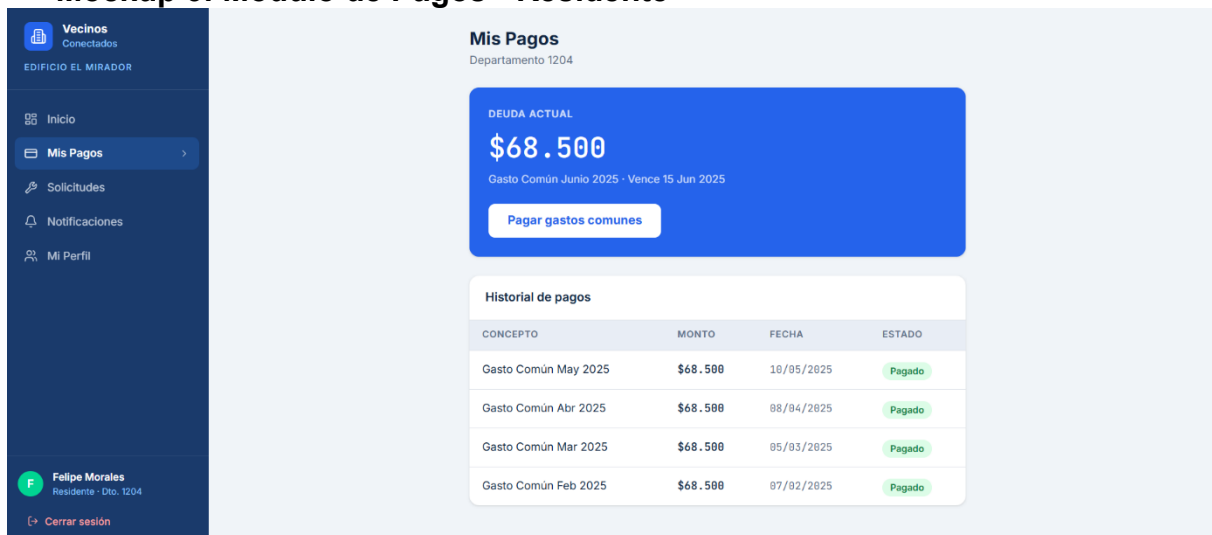
Esta pantalla concentra todas las notificaciones dirigidas al personal del edificio, incluyendo avisos sobre encomiendas recibidas, solicitudes de mantenimiento, recordatorios de pagos y comunicados emitidos por la administración. Las notificaciones se presentan ordenadas cronológicamente y es posible identificar aquellas que aún no han sido leídas, favoreciendo una comunicación oportuna entre la administración, los residentes y el personal operativo.

Mockup 5: Panel de Inicio – Residente



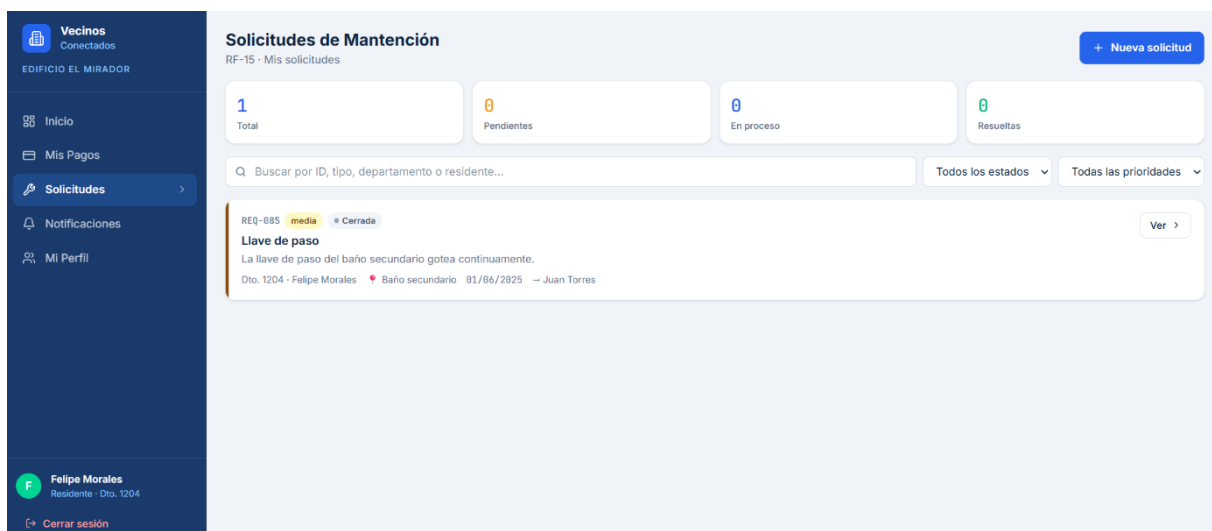
Esta interfaz corresponde al panel principal del residente después de iniciar sesión en la plataforma. La pantalla presenta un resumen de la información más relevante, incluyendo la deuda actual de gastos comunes, el número de solicitudes de mantención registradas y las notificaciones pendientes. Además, incorpora accesos directos para realizar el pago de gastos comunes, consultar solicitudes y revisar avisos recientes, permitiendo al residente acceder rápidamente a las funcionalidades de mayor uso.

Mockup 6: Modulo de Pagos - Residente



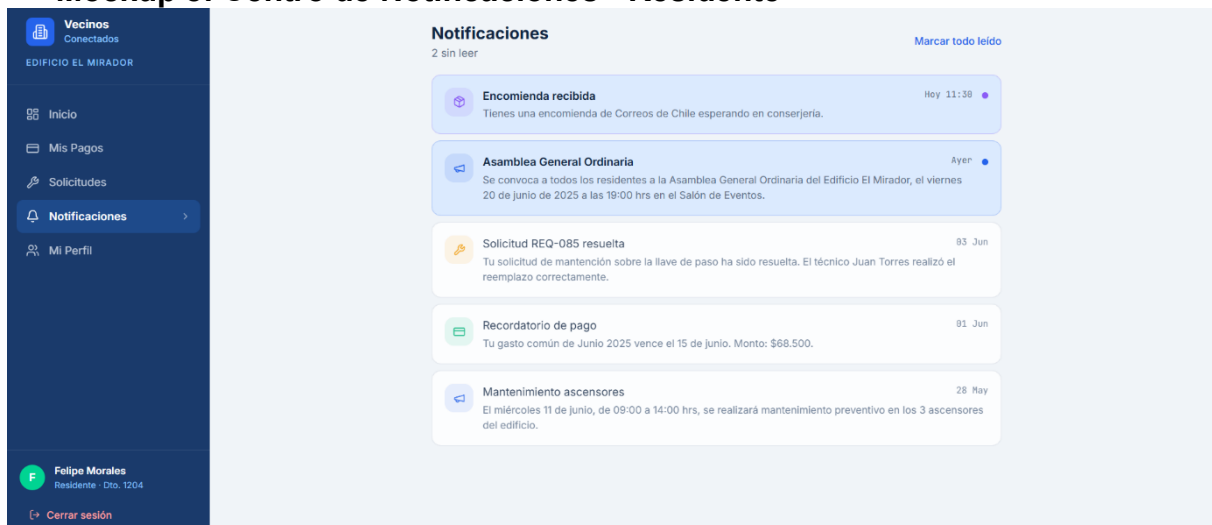
Esta pantalla permite al residente consultar el estado de su deuda de gastos comunes y acceder al proceso de pago mediante la plataforma integrada. También muestra un historial de pagos realizados, indicando el concepto, monto, fecha y estado de cada transacción. Esta funcionalidad facilita el seguimiento de los pagos efectuados y mejora la transparencia de la información financiera del departamento.

Mockup 7: Gestión de Solicitudes de Mantención - Residente



Esta interfaz permite al residente visualizar y administrar las solicitudes de mantención registradas para su departamento. La pantalla muestra un resumen de las solicitudes según su estado, además de herramientas de búsqueda y filtrado por estado y prioridad. Cada solicitud presenta información como el tipo de incidencia, descripción, ubicación, fecha de registro y estado actual, permitiendo al residente realizar un seguimiento del proceso de atención y registrar nuevas solicitudes cuando sea necesario.

Mockup 8: Centro de Notificaciones - Residente



Esta pantalla reúne todas las notificaciones dirigidas al residente, incluyendo recordatorios de pago, avisos de la administración, información sobre encomiendas recibidas y actualizaciones relacionadas con solicitudes de mantención. Las notificaciones se presentan de forma cronológica, diferenciando aquellas que aún no han sido leídas, lo que facilita el acceso oportuno a la información relevante del edificio.

Mockup 9: Perfil del Residente - Residente

Vecinos
Conectados

EDIFICIO EL MIRADOR

Inicio

Mis Pagos

Solicitudes

Notificaciones

Mi Perfil

Felipe Morales

Residente - Dto. 1204

Cerrar sesión

Mi Perfil

Editar perfil

Datos personales

Última actualización: 15 Mar 2025

NOMBRE COMPLETO
Felipe Morales García

RUT
12.345.678-9

TELÉFONO
+56 9 8765 4321

CORREO ELECTRÓNICO
f.morales@email.cl

TIPO DE RESIDENCIA
Propietario

MIEMBRO DESDE
Marzo 2021

Departamento asociado

NÚMERO
1204

PISO
12

SUPERFICIE
75 m²

% PARTICIPACIÓN
0.63%

Estado de cuenta

Deuda actual
\$68.500

Pagar ahora

Gasto Común Jun 2025 - Vence 15 Jun

Historial de pagos

CONCEPTO	MONTO	FECHA DE PAGO	ESTADO
Gasto Común May 2025	\$68.500	10/05/2025	Pagado
Gasto Común Abr 2025	\$68.500	08/04/2025	Pagado
Gasto Común Mar 2025	\$68.500	05/03/2025	Pagado

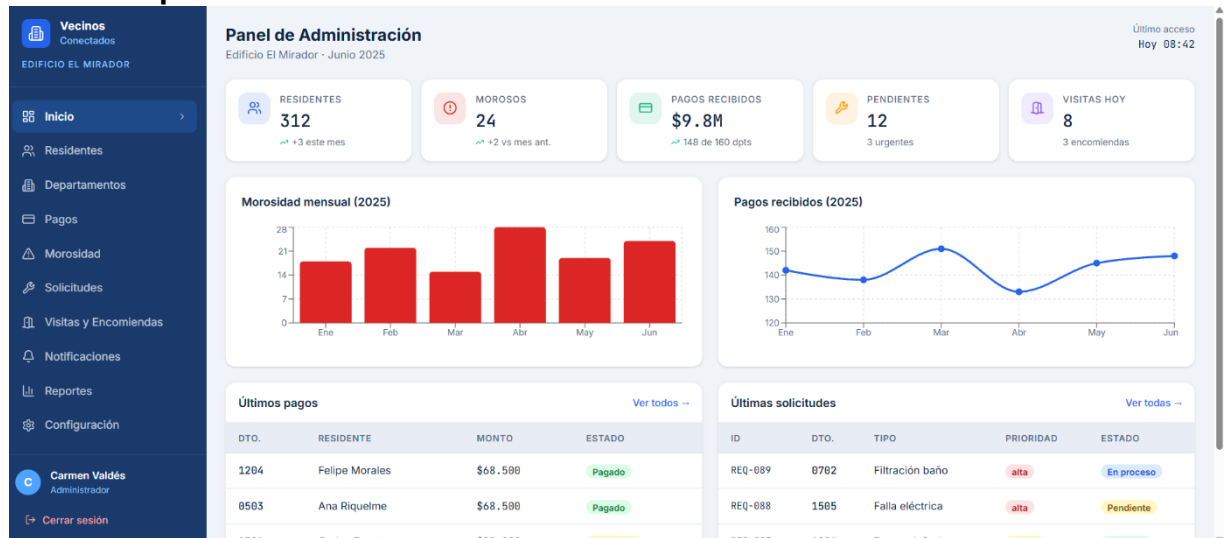
Mis solicitudes de mantención

Llave de paso
REQ-085 - 01/06/2025

Cerrada

Esta interfaz permite al residente consultar la información asociada a su cuenta y al departamento registrado en el sistema. Se muestran datos personales, información de contacto, tipo de residencia y antecedentes del departamento, como número, piso, superficie y porcentaje de participación. Además, incorpora un resumen del estado de cuenta y del historial de pagos, junto con un acceso para actualizar la información personal cuando sea necesario.

Mockup 10: Panel de Inicio – Administrador



Esta interfaz corresponde al panel principal del administrador del edificio. Presenta un resumen de los principales indicadores del sistema, como la cantidad de residentes registrados, departamentos con morosidad, pagos recibidos, solicitudes pendientes y visitas del día. Además, incorpora gráficos estadísticos y tablas con los últimos pagos y solicitudes registradas, proporcionando una visión general del estado operativo del edificio y facilitando la toma de decisiones.

Mockup 11: Gestión de Residentes - Administrador

Gestión de Residentes
8 residentes registrados

+ Agregar residente

Q. Buscar por nombre, RUT o departamento...

NOMBRE	RUT	DTO.	TIPO	TÉLFONO	CORREO	ESTADO	ACCIONES
F Felipe Morales	12.345.678-9	1204	Propietario	+56 9 8765 4321	f.morales@email.cl	Activo	
A Ana Riquelme	15.678.901-2	0503	Arrendatario	+56 9 7654 3210	a.riquelme@email.cl	Activo	
C Carlos Fuentes	9.876.543-2	1501	Propietario	+56 9 6543 2109	c.fuentes@email.cl	Activo	
L Laura Soto	18.234.567-8	0801	Arrendatario	+56 9 5432 1098	l.soto@email.cl	Moroso	
M Mario Vega	11.222.333-4	1102	Propietario	+56 9 4321 0987	m.vega@email.cl	Activo	
C Claudia Pinto	16.789.012-3	0604	Propietario	+56 9 3210 9876	c.pinto@email.cl	Inactivo	
R Roberto Muñoz	13.456.789-0	1803	Arrendatario	+56 9 2109 8765	r.munoz@email.cl	Activo	
P Patricia Lagos	17.890.123-4	0201	Propietario	+56 9 1098 7654	p.lagos@email.cl	Moroso	

Esta pantalla permite administrar la información de los residentes registrados en el sistema. El administrador puede consultar datos personales, departamento asociado, tipo de residencia, información de contacto y estado de cada usuario. Asimismo, dispone de herramientas para realizar búsquedas, aplicar filtros y registrar nuevos residentes, facilitando la administración de la comunidad.

Mockup 12: Gestión de Departamentos - Administrador

Vecinos Conectados EDIFICIO EL MIRADOR		Gestión de Departamentos Edificio El Mirador - 20 pisos - 160 departamentos				
		160 Total	136 Al día	12 Pendiente/Vencido	12 Morosos	
		Buscar por número o residente...			Todos los pisos	Todos los estados
DEPARTAMENTO	PISO	RESIDENTE	TIPO	ESTADO DE PAGO	% PARTICIPACIÓN	ACCIONES
0101	Piso 1	Felipe Morales	Propietario	Al día	0.63%	Ver detalle >
0102	Piso 1	Ana Riquelme	Arrendatario	Pendiente	0.63%	Ver detalle >
0203	Piso 2	Carlos Fuentes	Arrendatario	Vencido	0.63%	Ver detalle >
0204	Piso 2	Laura Soto	Propietario	Moroso	0.63%	Ver detalle >
0305	Piso 3	Mario Vega	Arrendatario	Al día	0.63%	Ver detalle >
0306	Piso 3	Patricia Lagos	Arrendatario	Pendiente	0.63%	Ver detalle >
0407	Piso 4	Roberto Muñoz	Propietario	Vencido	0.63%	Ver detalle >

Esta interfaz permite visualizar todos los departamentos registrados en el edificio junto con información relevante, como el residente asociado, tipo de ocupación, estado de pago y porcentaje de participación. Además, incorpora filtros de búsqueda por piso y estado, permitiendo consultar rápidamente el estado financiero y administrativo de cada departamento.

Mockup 13: Gestión de Pagos - Administrador



Vecinos

Conectados

EDIFICIO EL MIRADOR

Inicio

Residentes

Departamentos

Pagos

Morosidad

Solicitudes

Visitas y Encomiendas

Notificaciones

Reportes

Configuración

Carmen Valdés

Administrador

Cerrar sesión

Gestión de Pagos

Junio 2025

+ Generar cargo mensual

148

Pagados

8

Pendientes

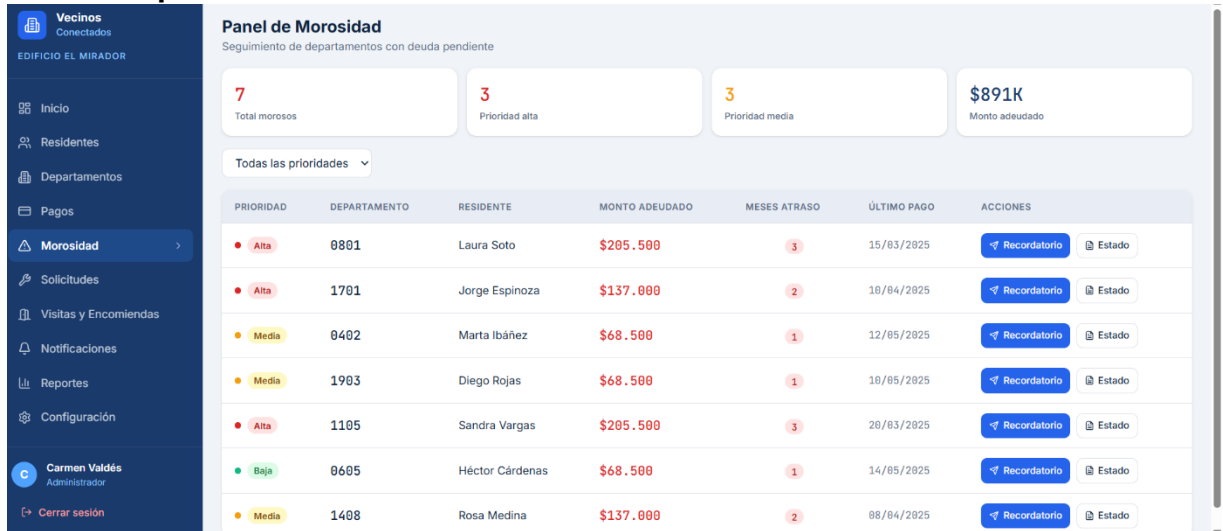
4

Vencidos

DEPARTAMENTO	RESIDENTE	CONCEPTO	MONTO	VENCIMIENTO	ESTADO	ACCIONES
1204	Felipe Morales	Gasto común Jun 2025	\$68.500	15/06/2025	Pendiente	Validar
0503	Ana Riquelme	Gasto común Jun 2025	\$68.500	15/06/2025	Pagado	
1501	Carlos Fuentes	Gasto común Jun 2025	\$82.000	15/06/2025	Pendiente	Validar
0801	Laura Soto	Gasto común May 2025	\$68.500	15/05/2025	Vencido	Validar
1102	Mario Vega	Gasto común Jun 2025	\$68.500	15/06/2025	Pagado	
0604	Claudia Pinto	Gasto común Jun 2025	\$68.500	15/06/2025	Pagado	

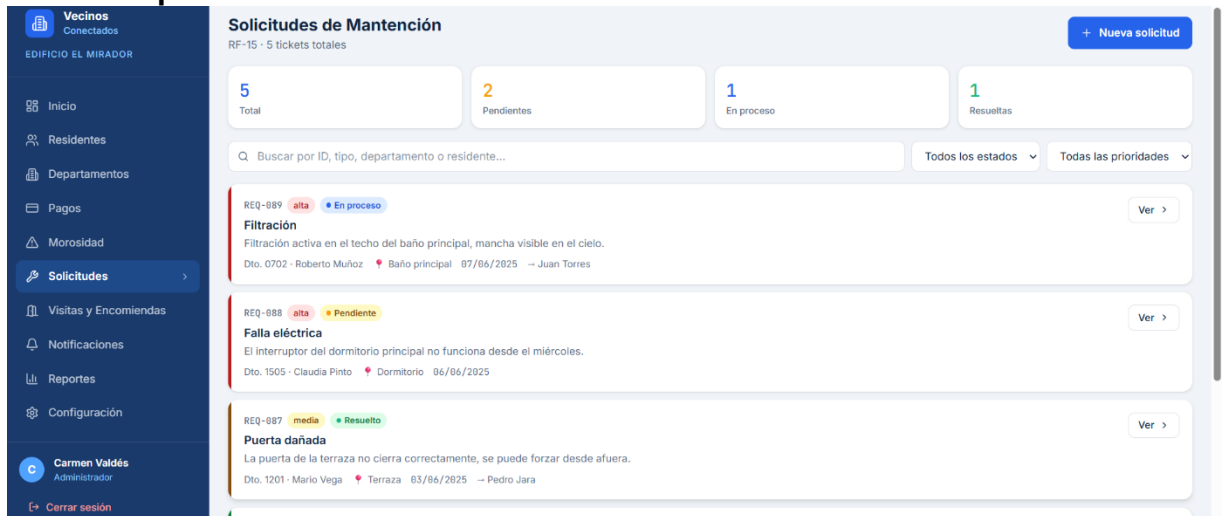
Esta pantalla permite administrar los pagos de gastos comunes registrados en el sistema. Se muestran indicadores sobre pagos realizados, pendientes y vencidos, junto con un listado que incluye departamento, residente, concepto, monto, fecha de vencimiento y estado del pago. Además, el administrador puede validar pagos y generar cargos mensuales para todos los departamentos.

Mockup 14: Panel de Morosidad - Administrador



Esta interfaz permite realizar el seguimiento de los departamentos con pagos pendientes. Se presentan indicadores sobre el número de residentes morosos, niveles de prioridad y monto total adeudado. El listado incluye información sobre el departamento, residente, deuda acumulada, meses de atraso y último pago realizado, además de opciones para enviar recordatorios o consultar el estado detallado de cada deuda.

Mockup 15: Gestión de Solicitudes de Mantenición - Administrador



Esta pantalla permite administrar todas las solicitudes de mantención registradas por los residentes. Se muestran indicadores del total de solicitudes y su distribución por estado, además de herramientas de búsqueda y filtrado. Cada solicitud incluye información sobre la incidencia reportada, prioridad, ubicación, residente solicitante y estado de atención, permitiendo al administrador supervisar el avance de los trabajos de mantenimiento.

Mockup 16: Gestión de Visitas y Encomiendas - Administrador

Vecinos
Conectados
EDIFICIO EL MIRADOR

Inicio
Residentes
Departamentos
Pagos
Morosidad
Solicitudes
Visitas y Encomiendas
Notificaciones
Reportes
Configuración

Carmen Valdés
Administrador
Cerrar sesión

Visitas y Encomiendas

Conserjería - Turno día

Registrar visita Registrar encomienda

8 Visitas hoy 1 En el edificio 2 Enc. pendientes 2 Retiradas hoy

Visitas Encomiendas

ID	VISITANTE	RUT	DTO.	RESIDENTE	INGRESO	SALIDA	AUTOR.
V-0091	Tomás Carrasco	20.111.222-3	1204	Felipe Morales	09/06 10:30	09/06 11:45	SI
V-0090	Gabriela Soto	15.333.444-5	0503	Ana Riquelme	09/06 09:15	-	SI
V-0089	Pedro Ramírez	12.555.666-7	0801	Laura Soto	08/06 14:00	08/06 15:30	SI
V-0088	Técnico GasNorte	-	1501	Carlos Fuentes	08/06 11:00	08/06 12:00	SI

Esta interfaz permite supervisar el registro de visitas y encomiendas del edificio. Se muestran indicadores relacionados con las visitas del día, personas presentes en el edificio y encomiendas pendientes o retiradas. Además, incorpora un historial detallado de los registros, permitiendo verificar la trazabilidad de los accesos y controlar la recepción y entrega de encomiendas.

Mockup 17: Centro de Notificaciones y Comunicados - Administrador

Vecinos
Conectados
EDIFICIO EL MIRADOR

Inicio
Residentes
Departamentos
Pagos
Morosidad
Solicitudes
Notificaciones
Reportes
Configuración

Carmen Valdés
Administrador
Cerrar sesión

Notificaciones y Comunicados

Envía avisos a residentes y personal

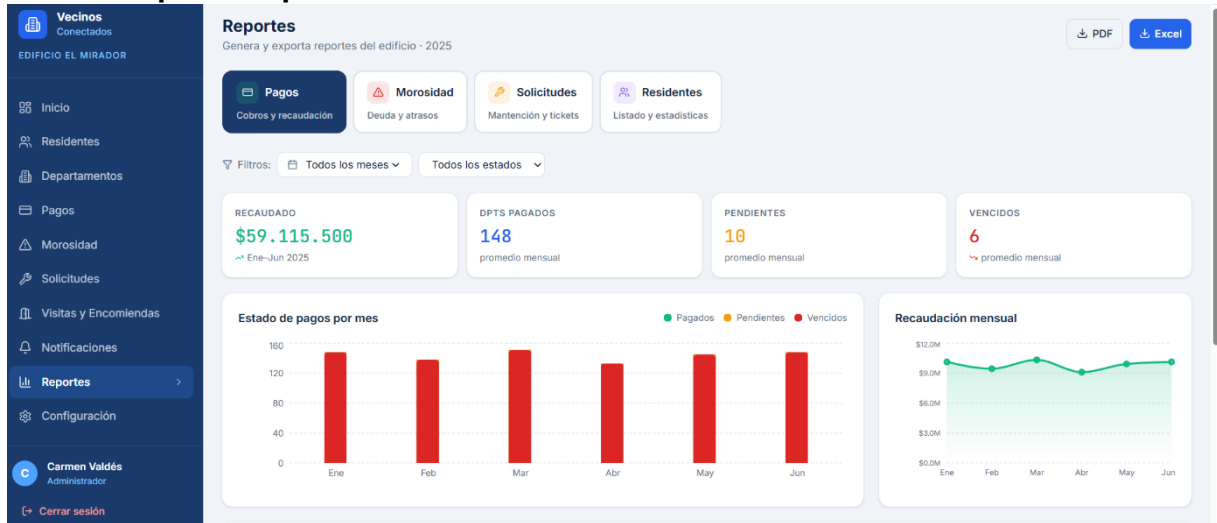
Nuevo comunicado

Comunicados enviados

Asamblea General Ordinaria Para: Todos los residentes	08 Jun 2025 alta
Mantenimiento ascensores Para: Todos los residentes	28 May 2025 media
Alza de gastos comunes Jul 2025 Para: Propietarios	25 May 2025 alta
Recordatorio de pago Jun 2025 Para: Con deuda pendiente	01 Jun 2025 media

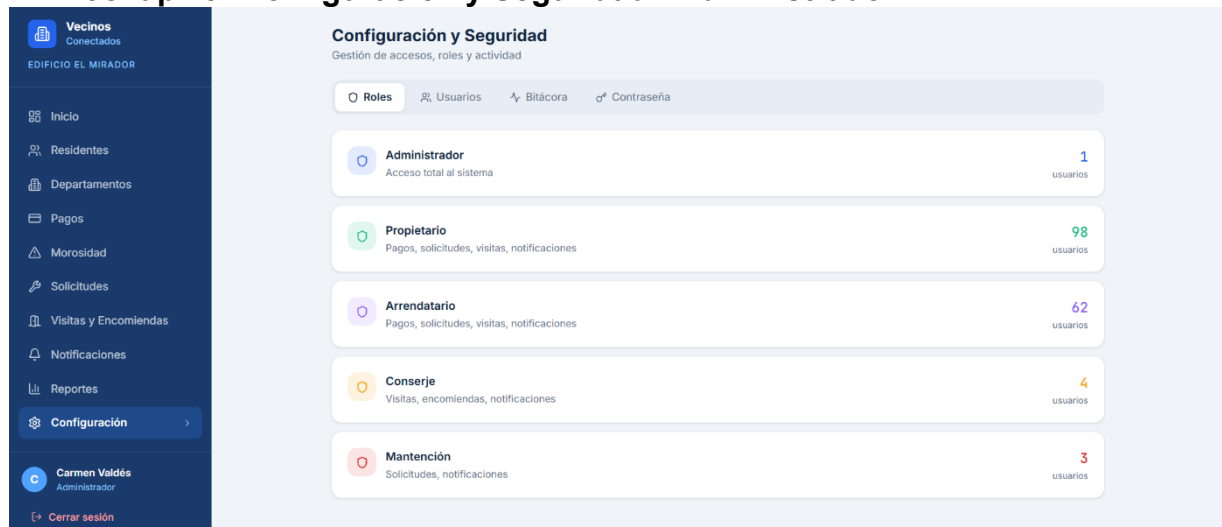
Esta pantalla permite gestionar los comunicados enviados a los residentes y al personal del edificio. El administrador puede consultar los avisos emitidos, identificar su prioridad y destinatarios, además de crear nuevos comunicados para informar actividades, recordatorios, mantenciones programadas u otras situaciones relevantes para la comunidad.

Mockup 18: Reportes - Administrador



Esta interfaz permite generar y consultar reportes administrativos relacionados con pagos, morosidad, solicitudes de mantención y residentes. La información puede visualizarse mediante indicadores y gráficos estadísticos, además de exportarse en formatos PDF y Excel para apoyar el análisis de datos y la elaboración de informes administrativos.

Mockup 19: Configuración y Seguridad - Administrador



Esta pantalla corresponde al módulo de configuración del sistema. Permite administrar los roles de usuario, gestionar cuentas, consultar la bitácora de actividad y configurar aspectos relacionados con la seguridad, como la administración de contraseñas y permisos de acceso. Este módulo garantiza un adecuado control de usuarios y la protección de la información del sistema.

7.3. Justificar herramientas de prototipado

Se seleccionó Figma como herramienta de prototipado para el desarrollo del sistema Vecinos Conectados, debido a que permite diseñar interfaces de alta fidelidad de forma colaborativa, facilitando la representación visual de las funcionalidades antes de la etapa de implementación del software.

La elección de esta herramienta se fundamenta en las siguientes características:

- **Prototipado interactivo:** Permite simular la navegación entre pantallas mediante botones, componentes interactivos y transiciones, ofreciendo una representación cercana al funcionamiento esperado de la aplicación final.
- **Diseño colaborativo:** Facilita el trabajo simultáneo entre los integrantes del equipo, permitiendo realizar modificaciones y recibir retroalimentación en tiempo real durante el proceso de diseño.
- **Componentes reutilizables:** Incorpora bibliotecas de componentes y estilos que permiten mantener una interfaz consistente, reutilizar elementos gráficos y agilizar el desarrollo del prototipo.
- **Diseño responsivo:** Permite diseñar y adaptar las interfaces para distintos tamaños de pantalla, verificando que la aplicación mantenga una correcta visualización y experiencia de usuario tanto en computadores como en dispositivos móviles. Esto contribuye al cumplimiento del requerimiento no funcional RNF-05, asegurando una interfaz accesible y adaptable para los diferentes perfiles del sistema.
- **Validación temprana del diseño:** Facilita la presentación del prototipo a los usuarios y al equipo de desarrollo para obtener retroalimentación antes de la implementación, permitiendo detectar mejoras de usabilidad y reducir modificaciones durante las etapas posteriores del desarrollo.

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

La evaluación heurística tiene como objetivo analizar la usabilidad del prototipo "Vecinos Conectados" mediante la aplicación de las 10 heurísticas de Jakob Nielsen. Esta evaluación permite identificar posibles problemas de interacción, navegación y experiencia de usuario antes de la implementación del sistema, contribuyendo a mejorar la calidad de la solución propuesta para administradores, residentes y personal del edificio.

8.2. Lista de verificación

DISTRIBUCION POR HEURISTICA					
Heurística	# Problemas	Sev. Max.	Sev. Prom.	Estado	Prioridad
H1 - Visibilidad del estado	5	0	1,8	MENOR	4 - Baja
H2 - Sistema y mundo real	1	0	1,0	MENOR	4 - Baja
H3 - Control y libertad	3	0	2,7	MENOR	4 - Baja
H4 - Consistencia y estandares	0	0	0,0	Sin problemas	Sin problemas
H5 - Prevencion de errores	1	0	3,0	MENOR	4 - Baja
H6 - Reconocimiento > recuerdo	0	0	0,0	Sin problemas	Sin problemas
H7 - Flexibilidad y eficiencia	1	0	2,0	MENOR	4 - Baja
H8 - Diseno minimalista	1	0	1,0	MENOR	4 - Baja
H9 - Recuperacion de errores	1	0	1,0	MENOR	4 - Baja
H10 - Ayuda y documentacion	3	0	1,7	MENOR	4 - Baja

8.3. Análisis y métricas de resultados

La evaluación heurística del prototipo fue realizada utilizando las 10 heurísticas de usabilidad propuestas por Jakob Nielsen. Durante la revisión se identificaron un total de 13 observaciones, clasificadas según su nivel de severidad.

Los resultados muestran que el sistema presenta un buen nivel general de usabilidad, destacando especialmente en consistencia visual, reconocimiento antes que memorización, validación de formularios y eficiencia de navegación. Los usuarios pueden acceder rápidamente a las funcionalidades principales mediante una interfaz organizada y coherente.

Las principales oportunidades de mejora se concentran en las áreas de ayuda y documentación, control del usuario sobre ciertas acciones y actualización inmediata de información relevante después de algunas operaciones. Asimismo, se detectó la ausencia de mecanismos de ayuda contextual y recorridos guiados para usuarios nuevos.

En términos generales, el prototipo cumple satisfactoriamente con la mayoría de las heurísticas evaluadas, permitiendo concluir que la solución propuesta ofrece una experiencia de usuario adecuada para los distintos perfiles contemplados en el sistema: administradores, residentes y personal del edificio.

9. CONTROL DE VERSIONES

9.1. Propósito

El control de versiones tiene como propósito registrar y gestionar las modificaciones realizadas durante el desarrollo del sistema Vecinos Conectados, permitiendo mantener un historial organizado de los cambios efectuados en el prototipo y en la documentación del proyecto. Esto facilita la trazabilidad de las versiones, el seguimiento de las mejoras implementadas y la recuperación de versiones anteriores en caso de ser necesario, contribuyendo a un desarrollo más ordenado y controlado.

9.2. Control de versión utilizado

Para el desarrollo del prototipo se adoptó el **versionamiento semántico**, ya que permite identificar de forma clara la evolución del diseño y de las funcionalidades implementadas mediante una estructura organizada de versiones. Este esquema facilita el seguimiento de los cambios realizados, la incorporación de nuevos requerimientos, la corrección de mejoras y la recuperación de versiones anteriores cuando es necesario. Durante el proceso de diseño se registraron distintas iteraciones del prototipo, documentando los avances realizados en cada una de ellas.

Durante el desarrollo del prototipo se registraron las siguientes versiones:

Versión	Fecha	Descripción
V.1	09/06/2026	Creación del prototipo base e implementación de las instrucciones iniciales del proyecto.
V.2	09/06/2026	Incorporación del diseño responsivo para dispositivos móviles y escritorio.
V.3	16/06/2026	Implementación de los requerimientos funcionales RF-15.
V.4	16/06/2026	Implementación de los demás requisitos funcionales definidos para el sistema.
V.5	16/06/2026	Ajuste del color de la pestaña y mejoras visuales de la interfaz.
V.6	17/06/2026	Actualización de la pantalla de cambio de contraseña.
V.7	17/06/2026	Implementación de la validación de contraseña.
V.8	17/06/2026	Validación del campo de descripción de solicitudes
V.9	17/06/2026	Incorporación de la visualización de solicitudes en el perfil del residente.
V.10	17/06/2026	Se agregó la obligatoriedad de la descripción en las solicitudes al asignarlas al personal y se realizaron ajustes finales al prototipo.

9.3. Justificar herramientas de versionamiento

Se utilizó GitHub como herramienta de control de versiones debido a que permite almacenar el proyecto en un repositorio remoto, registrar el historial de cambios mediante commits, administrar diferentes versiones del sistema y facilitar el trabajo colaborativo entre los integrantes del equipo. Además, GitHub permite recuperar versiones anteriores,

comparar modificaciones realizadas y mantener una trazabilidad completa del desarrollo del proyecto. Para el prototipo desarrollado en Figma, también se aprovechó el historial de versiones de la herramienta, permitiendo documentar cada iteración y mejora realizada durante el proceso de diseño.

10.CONCLUSIONES

El desarrollo del Documento de Arquitectura de Software permitió definir una solución estructurada para el sistema **Vecinos Conectados**, utilizando una arquitectura monolítica en capas junto con el patrón MVC y el modelo de vistas 4+1. Estas decisiones facilitaron la organización de los componentes, la representación de los procesos del negocio y la trazabilidad de los requerimientos funcionales y no funcionales, estableciendo una base sólida para el desarrollo e implementación del sistema.

La incorporación de prototipos de alta fidelidad, principios de diseño de software, criterios de calidad y herramientas de control de versiones permitió validar la propuesta antes de su implementación. En conjunto, estos elementos contribuyen a que la solución sea mantenible, segura y escalable, ofreciendo una plataforma capaz de mejorar la administración del edificio, optimizar la gestión de los gastos comunes y fortalecer la comunicación entre residentes, administración y personal.

11. BIBLIOGRAFÍA

- Figma. (2026). *Figma Help Center*. Recuperado de <https://help.figma.com>
- GitHub. (2026). *GitHub Docs*. Recuperado de <https://docs.github.com>
- Object Management Group. (2017). *Unified Modeling Language (UML), Version 2.5.1*. recuperado de <https://www.omg.org/spec/UML/>
- Len Bass, Paul Clements, & Rick Kazman. (2022). *Software Architecture in Practice* (4.^a ed.). Addison-Wesley.
- Jakob Nielsen. (2020). *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. Recuperado de <https://www.nngroup.com/articles/ten-usability-heuristics/>

12. ANEXOS

- Diagrama de Casos de Uso:
- Diagrama de Clases: [Diagrama de Clases.mdj](#)
- Diagrama de Componentes: [diagrama de componentes.mdj](#)
- Diagrama de Paquetes: [Diagrama de Paquetes.mdj](#)
- Diagrama de Actividad: [diagrama de actividadd.mdj](#)
- Diagrama de Despliegue: [Diagrama de Despliegue IngSoftware.mfj](#)
- Evaluación heurística de Nielsen:
https://1drv.ms/x/c/bd6b66f386c07886/IQCTDakEHYmRQq5KJitX3_QAAadusegeagJtWhPDLs-JohxY
- Plantilla registro de pruebas de prototipo:
https://1drv.ms/x/c/bd6b66f386c07886/IQAsZ1xloPE4Q4B_F7SDfj9xAY3sMvJRbh-ZYZju6X-WkvM
- Link GitHub: https://github.com/RocioBustoss/Vecinos_Conectados_Versiones

13. CARTA GANTT

- Link carta Gantt:
https://1drv.ms/x/c/bd6b66f386c07886/IQDy3Ah42ITtSqAFTk7-3ySVAWK-wbmKnPv_Int6S0ygRK8